

Masters Project: Final Thesis Report

DyNNSoC: Efficient Acceleration of Dynamically-Quantised Neural Networks using Bit-Serial Arithmetic and a RISC-V Controller

Thomas O'Dea

Student ID: 21386441

Supervisor: Dr. Deepu John

A thesis submitted in part fulfilment of the degree of

M.E. in Electronic and Computer Engineering



UCD School of Electrical and Electronic Engineering

University College Dublin

April 19, 2026

SCHOOL OF ELECTRICAL & ELECTRONIC ENGINEERING

HAZARD IDENTIFICATION FORM

**ME BIOMEDICAL ENGINEERING, ME ELECTRICAL POWER ENGINEERING,
ME ELECTRONIC & COMPUTER ENGINEERING,
BE ELECTRICAL ENGINEERING OR BE ELECTRONIC ENGINEERING**

Please complete the form using **BLOCK CAPITALS**. Note that you will **not** be allocated a space in the project lab (if required) until you supply a fully signed off version of this form.

Student Name:	Thomas O'Dea	Student Number:	2138 6441		
Email:	thomas.odea1@ucdconnect.ie				
Project Title:	Neural Network Accelerator using RISC-V				
Programme:	Computer & Electronic Engineering	BE		ME	X

Health and Safety Aspects

Identify any potential safety hazards for your project, e.g. dangerous voltages, rotating machinery, radiation, soldering fumes, sitting in front of a computer for prolonged periods

- Prolonged periods seated at a computer and associated risks:
 - Repetitive strain injuries e.g. carpal tunnel syndrome
 - Eye strain

Indicate how all safety hazards identified above will be managed to achieve a safe working environment for your project

Use of (my own) ergonomic keyboard.
As part of my placement I have completed office ergonomics training.

Indicate procedures in place for on-campus activities relating to the project, particularly in relation to unsupervised access to the project laboratory

The project is largely computer-based and will not incur significant health and safety hazards. However, I will need to program FPGA's and will need a working space to keep the board and a PC to connect it to.

Indicate how project activities and final objectives will be revised, if appropriate, if there is an extended lockdown period on the UCD campus

If necessary it is possible to configure Vivado as a remote server for FPGA configuration. This would mean some amount of work can be done by remote. My workspace on campus will then be used to house a permanently-on PC as a remote server, connected to the FPGA board being used.

Ethical and Sustainability Aspects

Identify any ethical considerations which may affect the progress of your project.

All novel research carries some level of ethical risk with respect to its potential applications. However, this project intends only to improve the energy/cost efficiency of existing machine learning technologies by improving the hardware available for training and inference. Consequently it has minimal ethical ramifications.

Identify any sustainability considerations which may affect the progress of your project.

The project intention is to design a hardware accelerator which will be considerably more energy efficient than existing solutions. In this regard the project promotes sustainable energy use. This is unlikely to affect the project during the year.

Student Project Laboratory Access

Laboratory desk space is **limited**. Each allocated desk includes **one monitor (no additional monitors are available), one HDMI cable, and a power outlet** for your laptop connection.

A small number of desks have **ethernet connections**, but these are also limited.

Wi-Fi is available. Students are advised to connect to the **eduroam** network for safety and stability.

Students are advised to bring their own HDMI–USB-C adapters, as the laboratory does not provide them.

If a PC is required, it must be requested by the project supervisor with proper justification. **The number of PCs is limited and restricted.**

Do you require a desk in the project laboratory?	Yes	X	No	
Will your supervisor instead provide a desk for you?	Yes		No	X
If so, please indicate the location				

Supervisor:	Deepu John	Signature <i>Deepu John</i>
-------------	------------	--------------------------------

Submit the completed form to your project module on Brightspace before the end of week 3 of the Autumn trimester.

In addition, email the completed form to the Head of the Electronic Workshop, Andrea Ortiz-Cuadros (andrea.ortizcuadros@ucd.ie)

Abstract

The drastic improvement in the capabilities of artificial intelligence in the last decade has been underpinned by the growing scale of neural network models. However, the computational demands of these models have also grown, leading to increased energy consumption and environmental impact. Dedicated hardware acceleration for neural networks offers efficiency gains in performance, power, and silicon area.

This project approaches neural network acceleration by focusing on *dynamically-quantised* neural networks, that is networks where the precision of the parameters, input data, and intermediate results can be dynamically adjusted according to the needs of the task at hand. The precision may be adjusted according to internal factors (such as the complexity of the input data) or external factors (such as the current load on the system).

This work presents DyNNSoC, a system-on-chip designed to accelerate dynamic neural networks on AMD FPGA hardware. The design integrates an array of processing elements for efficient neural network inference with a lightweight general-purpose CPU for flexible control. A firmware API facilitates deployment of convolutional neural networks on the system. Previous works are explored which study from a theoretical standpoint the impact of dynamic quantisation on neural network performance, and this work provides an implementation, demonstrating the conceptual viability of dynamic quantisation.

Lay Abstract

Neural networks are a family of computational algorithms originally popularised in the 1960s. Conceptually, they are relatively straightforward, being composed of many individual ‘neurons’ which are each very simple in their operation. However, when combined into a large structure, neural networks can recognise extremely complex patterns in data. They can be used to ‘predict’ appropriate data, allowing them to perform tasks such as image recognition by ‘predicting’ the appropriate label to place on an image, or even generating text by ‘predicting’ the appropriate next word in a sequence of words.

Neural networks have been found to improve their performance drastically as their scale increases – usually summarised in a single ‘parameter count’. As our computational capabilities have grown exponentially, the capabilities of neural networks has grown too. Today, they are the algorithm that underpins the modern revolution in artificial intelligence including large-language models like ChatGPT.

In a positive feedback sense, this improvement in AI capabilities has led to increased demand for larger and more powerful networks, which demand more computational resources.

Typical computation tasks are performed using a general-purpose CPU (‘central processing unit’). While modern consumer CPUs are highly sophisticated devices, they are ultimately designed for *flexibility*, allowing consumers to run a wide range of software on a single device. This flexibility comes at the expense of efficiency.

Hardware dedicated to a specific class of algorithms can offer enormous efficiency gains, and is justified when a single class of algorithms demand a large amount of computation. In this project, new hardware is designed to accelerate neural networks, with a focus on neural networks that can dynamically adapt their complexity according to the needs of the task at hand. The designed system combines an array of ‘processing elements’ for highly efficient neural network inferences with a lightweight general-purpose CPU for flexible control.

Contents

Hazard Identification Form	1
Abstract	4
Lay Abstract	5
Acronyms	8
 I Background	 9
1 Introduction	10
1.1 Preamble	10
1.2 Primer on Neural Networks	11
1.3 Dynamic Neural Networks	11
1.4 Primer on RISC-V Instruction Set	13
 2 Review of Existing Literature	 14
2.1 Neural Network Quantisation	14
2.2 Dynamic Neural Networks	15
2.2.1 Adaptation Signals	15
2.2.2 Types of Dynamic NN Adaptation	16
2.3 Dynamic Quantisation	17
2.4 Hardware Acceleration	17
 3 Existing Processing Element Design	 18
3.1 Bit-Serial Arithmetic	18
3.1.1 Motivation	19
3.1.2 Bit-Serial Adder	19
3.1.3 Bit-Serial Multiplier	20
3.1.4 Transposed Memory Format	23
3.2 Existing MVU Design	24
3.2.1 High-Level Operation and Registers	24
3.2.2 Datapath	25
3.2.3 Data Banks	26
 II Work Completed	 27
4 Processing Element	28

4.1	Modifications to the MVU	28
4.1.1	Bug fix: Data banks 1-31 unusable	28
4.1.2	Bug fix: Race conditions in sequential logic designs	30
4.1.3	Register Map Redesign	31
4.1.4	Memory Interface Redesign and AHB Integration	32
4.2	Verification	37
5	System Integration	38
5.1	SoC Architecture	38
5.1.1	MVU Array	40
5.1.2	Ibex: RISC-V SoC Core	40
5.1.3	RAM	41
5.1.4	GPIO	41
5.1.5	ROM Loader	42
5.1.6	Instruction Memory	42
5.2	EDA Workflow	43
5.2.1	Languages, Compilers, and Simulators	44
5.2.2	Use of FuseSoC	45
5.2.3	Use of MVU Stub Files	47
5.3	AHB Infrastructure	47
5.3.1	SystemVerilog Interface	47
5.3.2	AHB Interconnect	48
5.3.3	Register Map Utilities	49
5.4	Memory Map	50
5.5	DMA Design	51
5.5.1	Design	51
5.5.2	Verification	54
5.6	AHB Arbitration	55
6	System Review	59
6.1	Integration Testing	59
6.2	FPGA Implementation	60
6.3	Power Estimation	61
6.4	Neural Network Firmware	63
6.4.1	C API	63
6.4.2	Proof of Concept	64
III	Conclusion	66
7	Results and Research Contribution	67
7.1	FPGA Resource Utilisation	67
7.2	Power Consumption	67
7.3	Discussion	69

8	Suggestions for Future Work	70
8.1	Scope for Technical Improvements	70
8.2	Scope for Continued Research	71
9	Ethics and Sustainability	71

Acronyms

NN	Neural Network
DyNN	Dynamic Neural Network
MVU	Matrix-Vector Unit
AGU	Address Generation Unit
SoC	System-on-Chip
AHB	Advanced High-performance Bus
DMA	Direct Memory Access
DMAC	Direct Memory Access Controller
SAIF	Switching Activity Interchange Format
IRQ	Interrupt Request

Part I

Background

Chapter 1

Introduction

1.1 Preamble

Neural networks (NNs) have underpinned a modern revolution in artificial intelligence. Their training and inference are highly compute-intensive tasks, but involve extremely repetitive and parallelisable operations which are inefficient on general-purpose CPUs. This inefficiency has costly implications not just in the computational resources it demands, but increasingly in the environmental cost of the energy consumed by large-scale AI services. Further, this large energy consumption necessitates large cooling systems, which has further environmental impact and infrastructural overhead. For these reasons and many more, neural networks are a natural target for hardware acceleration.

Dedicated hardware acceleration for a specific algorithm class offers efficiency gains in performance, power, and silicon area. While difficult to quantify in the most general sense, there is no question that dedicated hardware designs are more than justified for the scale of neural network workloads being run today.

1.2 Primer on Neural Networks

A basic understand of the computational needs of neural networks is necessary to understand this project.

A basic neural network can be viewed as a series of ‘layers’. At each layer, there is an *input vector* (or larger tensor) which undergoes two operations in the layer:

- A linear operation on the whole vector, equivalent to a matrix multiplication
- A non-linear operation acting on individual elements of the vector

Note that this description does not cover every network architecture available, but does cover a large portion of them.

The non-linear function is most popularly *ReLU*, a simple piecewise function $f(x) = \max(0, x)$. Other non-linear functions are not considered in this project, since ReLU is extremely widely used owing both to its straightforward implementation and superior performance.

The linear operation is often the most computationally tasking, requiring matrix multiplication on very large vectors, repeated for many layers (anywhere from 2 to 1,000, depending entirely on the network in question). Fortunately, matrix multiplication is highly parallelisable. This is because the results in each element of the output vector do not depend on one another, only on the input vector and the matrix. This motivates us to design a hardware processing element that can take advantage of this parallelism to efficiently compute matrix-vector products. The ReLU activation function is integrated into this design.

1.3 Dynamic Neural Networks

In this project, the focus is on accelerating a specific class of neural networks: dynamic neural networks (DyNNs). Consider the result in table 1.1, which is taken from [20]. Naturally, larger, more computationally expensive model yield improved performance. However,

there are clear diminishing returns: a 40% increase in compute time between MobileNetV3 and EfficientNetV2 yields only a 1.2% (percentage-point) increase in accuracy. As accuracy approaches the high nineties, the cost of further improvement becomes increasingly prohibitive. Similar studies identify this phenomenon within a single family of network that share a common architecture and only differ in depth [9].

Model	Accuracy (%)	F1 Score	sec/inference	GFLOPs	Model Size (MB)
MobileNetV3	89.62	0.8889	0.000113	0.01	6.18
ResNet18	84.47	0.8432	0.00005	0.04	42.83
EfficientNetV2	90.82	0.9104	0.000157	0.01	77.46
SqueezeNet	62.24	0.6198	0.000048	0	2.95
ShuffleNetV2	89.21	0.9027	0.000127	0	5.17

Table 1.1: CIFAR-100 Model Performance (taken from [20] originally table 2)

Neural network creators are faced with a tradeoff between limited computational resources and a desire for improved accuracy. Dynamic neural networks address this issue by allowing the network to adapt its structure and computational cost according to the complexity of the input sample. Consider, for example, the task of image classification exemplified in figure 1.1.

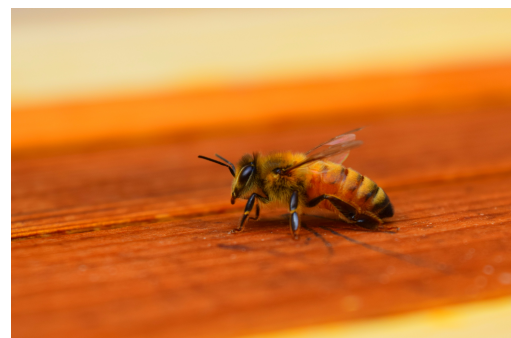


Figure 1.1: The bee on the left is set against a clear, white background and has clear, prominent black-and-white stripes. The bee on the right is smaller in the image, has less obvious stripes, and less contrast with the background. A smaller, simpler, cheaper image recognition model may be sufficient to classify the left image than would be needed for the right image.

Dynamic networks may adapt their depth, width, or resolution to adjust their computa-

tional cost. These ideas are explored in section 2.2.

This work presents DyNNSoC, a system-on-chip targetted at dynamic neural network acceleration. It is intended for FPGA Implementation on Xilinx hardware. The rest of this report is structured as follows:

- chapter 2 reviews the existing literature surrounding dynamic neural networks, dynamic quantisation, and hardware acceleration for these algorithms.
- chapter 3 gives an overview of an existing design for a processing element presented in [4], highlighting its advantages and issues.
- chapter 4 discusses how the existing design was fixed, modified, and extended for this project.
- chapter 5 discusses how this processing element is integrated into a fully functional system, including several newly designed components.
- chapter 6 discusses how the designed system was implemented for FPGA and tested.
- chapter 7 presents the results.
- chapter 9 briefly comments on the implications this project has for sustainability and ethical matters.
- chapter 8 suggests further developments for the project or future research.

1.4 Primer on RISC-V Instruction Set

RISC-V is the most popular open-standard instruction set in the world. It defines a set of instructions for CPUs to implement that allow compiled binaries and compiler toolchains to be shared across a wide range of processors, which may have different physical implementations from one another. “RISC” refers to ‘reduced instruction set computing’, a philosophy

where the individual instructions are relatively straightforward. RISC-V is a load/store architecture, meaning arithmetic and logic operations are only possible on values stored in the file of 32 CPU registers. There are a limited number of ‘load’ and ‘store’ instructions for moving data to/from the CPU registers to/from main memory.

RISC-V is noted for its use in academia due to the availability of open-source implementations and toolchains. However, it is also beginning to see increasing popularity in industry as low-power design is becoming a priority, and RISC’s simplistic design makes it ideal for reduced power. This project makes use of a small, lightweight RISC-V core which is introduced in more detail later.

Chapter 2

Review of Existing Literature

2.1 Neural Network Quantisation

A class of neural network known as *quantised* NNs are targeted for acceleration in this report. The mathematical foundations of NNs are generally studied as real numbers assuming infinite precision. This infinite precision has historically been approximated by floating-point representations, typically 32 bits. However, there is abundant evidence (e.g. [27], [10]) to suggest that NNs are tolerant to reduced precision and can achieve similar performance using much lower precision fixed-point representations. A Google whitepaper from 2018 surveys this area [13]. This process, known as quantisation, saves power due to the overall reduction

in data movements and storage.

Quantised networks may have *mixed precision* wherein different layers are calculated at different bitwidths. One approach for optimising these bitwidths using reinforcement learning is discussed in [23].

Later, the challenges associated with implementing this in hardware are discussed.

2.2 Dynamic Neural Networks

While NNs typically perform a set series of computations (alternating layers of linear transformations and non-linear functions), *dynamic* NNs (DyNNs) are a class of NNs which adapt their structure or parameters on a per-input basis, allotting more computational resources according to some signal.

A 2021 survey [8] of DyNN types reviews two features of dynamic NNs: the nature of the dynamic variation in inference, and the signal used to determine adaptation.

2.2.1 Adaptation Signals

Various signals may be used to indicate how much to dynamically adjust the complexity of a neural network. The survey [8] only considers data-dependent signals for adaptation (that is, signals which depend on the sample on which inference is being run), categorising them into three groups:

- Confidence-based - a measure of confidence is established from intermediate computations. For example, [22] evaluates the entropy of the output at an early stage and determines whether a confidence threshold is met to allow early exiting.
- Policy networks - a separate, smaller network is trained to analyse samples to decide dynamic parameters of the main network. For example, [25] uses a policy network to decide which blocks of a ResNet to ‘drop’. This idea was subsequently explored in [15] as discussed below.

- Gating functions - a more general class of methods where distinct parts of the networks may be adapted according to different signals.

Notably, the survey does not discuss non-data-dependent signals for adaptation. These are discussed somewhat in [7]. These could include:

- Battery level available on an edge computing device
- Peripheral sensor data (e.g. light levels may indicate the computational resources necessary to accurately classify images)
- Size of backlog of samples to run inference
- etc.

This report will focus on accelerator designs which accommodate both categories of signal - it is therefore necessary to provide **maximum flexibility** via a **programmable RISC-V controller**.

2.2.2 Types of Dynamic NN Adaptation

The nature of the dynamic variation in a NN could include:

- Varying the depth of the network (i.e. early exiting as seen in [22] and [24])
- Varying the width of the network (as in [26])
- Varying the network architecture itself (e.g. dynamic routing) (as in [19])
- Varying the precision of operations (i.e. dynamic quantisation)

The last of these points is least well-explored and will be the focus of our research. A number of existing works in this space are discussed in the section that follows.

2.3 Dynamic Quantisation

[11] was a 2020 paper which was the first to consider in detail the accuracy-efficiency tradeoff offered by dynamic quantisation. It notes savings in memory, latency, and energy consumption in return for a small tradeoff in model performance. No insight is given regarding appropriate signals to determine levels of quantisation, but a framework is established for applying the quantisation function and its impact on accuracy is studied. Resource savings are quantified in terms of total bit operations, and the paper demonstrates that, using dynamic precision, MobileNetV1, MobileNetV2, and ResNet50 can see a 73%, 72%, or 70% reduction in bit operations (respectively) with a $<3\%$ reduction in top-1 accuracy.

Similar ideas are independently explored in [15] which goes further to propose a separate policy network for deciding precisions at each bitwidth. Rather than contrasting performance with a full-precision model, it compares the dynamic model with a statically quantised model of equivalent computational cost, demonstrating a 2% improvement in top-1 accuracy for equal computational resources.

2.4 Hardware Acceleration

A 2024 survey [17] outlines the state of NN hardware accelerators available in academic resources. While the survey identifies quantised NN's as a target model-class for hardware acceleration, there is no discussion of the specific requirements of *dynamically quantised* networks or indeed mixed-precision at all.

Overlooked in this survey was a 2023 accelerator *BARVINN* [3], which fills the gap for a dynamic-precision accelerator. BARVINN combined a processing element design presented in [4] with a RISC-V controller presented in [2]. The RISC-V controller allows for software-programmable network architectures with flexible behaviour, while the processing elements employ a bit-serial architecture for multiply-accumulate (MAC) operations, allowing software-controllable tradeoff between precision and performance.

Unfortunately, while [3] demonstrates the conceptual viability of the platform, the source code in both [4] and [2] have numerous HDL bugs, and only a fledgling software toolchain. The code base is open-source.

The processing element [4] was originally created for *mixed* precision NN models and not necessarily *dynamic* precision, however its architecture was determined to be suitable with modifications.

Chapter 3

Existing Processing Element Design

In this chapter, the fundamental processing element used to perform neural network inference in [4] is presented. The chapter opens with a primer on bit-serial arithmetic, which is a key technique used in the design of this processing element. An overview of the design in [4] is then given, noting some errors and limitations. These limitations are addressed in this project as outlined in the Work Completed part.

3.1 Bit-Serial Arithmetic

Disclaimer: The hardware required to implement the bit-serial arithmetic techniques is already available from [4] – its design is not a part of this project, but understanding it is imperative to appreciating the efficiency gains offered by bit-serial arithmetic for dynamic-precision calculations.

3.1.1 Motivation

Conventionally, arithmetic operations are performed in hardware using digital buses sized according to the operands. For instance, 32-bit multiplier will have two 32-bit inputs, feeding a matrix of 1,024 AND gates, then a (costly) cascade of adders to give a 64-bit output. This architecture allows calculations in a single clock cycle, at the expense of large amounts of hardware.

Critically, a structure like this has fixed operand widths implied by the width of the input buses. This is problematic for a mixed-precision system, where the precision of operands may vary. The hardware must be sized according to the largest operands, where smaller operands will be padded with zeros. The smaller operands will consume less dynamic power due to reduced switching activity, but the static power consumption of the unused hardware will be wasteful, and there is a large amount of hardware being wasted. In an ASIC flow, this wasted hardware implies larger silicon area, with an associated cost. In an FPGA flow, the wasted hardware increases the utilisation of the FPGA's resources.

3.1.2 Bit-Serial Adder

Bit-serial arithmetic is a powerful alternative technique. In bit-serial arithmetic, only one bit from each operand is taken as input, and one output bit computed. For example, figure 3.1 shows a simple bit-serial adder. A shift register at each input feeds one bit at a time, and a shift register at the output stores the output bits as they are computed. The carry from each bit addition is stored in a 1-bit register and fed into the next bit addition on the following cycle. This architecture allows for variable precision, as the number of bits fed into the adder can be varied according to the precision required by the operands. The hardware is significantly smaller. Calculations with larger operands require more clock cycles, allowing a trade-off between precision and latency during operation.

It is also possible to avoid the need for shift registers to serialise/deserialise the inputs/outputs by simply storing operands in memory addressable by bit. This is discussed in

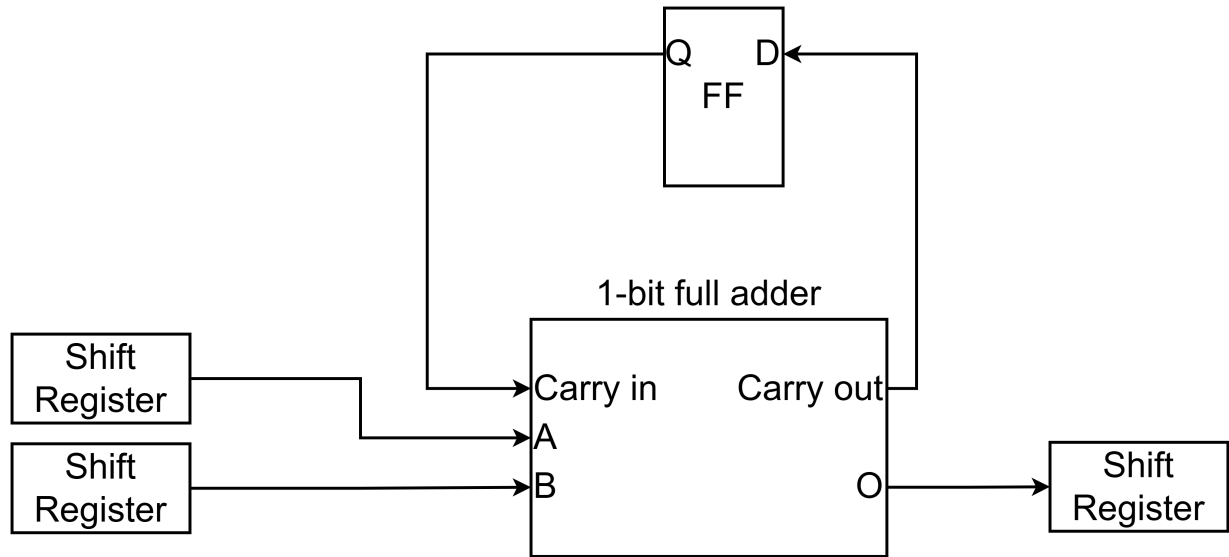


Figure 3.1: Example minimal implementation of a bit-serial adder. The input operands are fed bit-by-bit beginning with the least significant bit. Overflows are stored in a 1-bit ‘carry’ flop and fed into the next most significant bit on the following cycle. The output is fed into a shift register, again beginning with the least significant bit. Control logic (e.g. resets and load signals for the shift registers) are omitted for readability.

more detail in the next section.

3.1.3 Bit-Serial Multiplier

Bit-serial multiplication presents a slightly more complex challenge. Let x, y be expressed two 3-digit binary values:

$$x = 8x_3 + 4x_2 + 2x_1 + x_0 \quad x_0, x_1, x_2, x_3 \in \{0, 1\} \quad (3.1)$$

$$y = 8y_3 + 4y_2 + 2y_1 + y_0 \quad y_0, y_1, y_2, y_3 \in \{0, 1\} \quad (3.2)$$

Then their product is given by:

$$z = xy \quad (3.3)$$

$$= (8x_3 + 4x_2 + 2x_1 + x_0)(8y_3 + 4y_2 + 2y_1 + y_0) \quad (3.4)$$

This is generalised by:

$$z = \sum_{i=0}^{N-1} \sum_{j=0}^{N-1} 2^{i+j} x_i y_j \quad (3.5)$$

where N is the width of the operands in bits.

A naïve implementation might approach this calculation like a nested **for** loop, iterating over each combination of bits from x and y , multiplying them and multiplying by the appropriate power of two, then accumulating the result in an accumulator register. Recall that one-bit multiplication is simply an AND gate, and multiplication by a power of two is simply a left-shift operation, making it a relatively inexpensive approach. However, it has the disadvantage of requiring the accumulator register to support addition of values with a wide range of magnitudes.

[4] presents a more efficient approach. Equation 3.4 can be expanded and rewritten as:

$$\begin{aligned} z = & 64[x_3y_3] + 32[x_3y_2 + x_2y_3] + 16[x_3y_1 + x_2y_2 + x_1y_3] + \\ & 8[x_3y_0 + x_2y_1 + x_1y_2 + x_0y_3] + 4[x_2y_0 + x_1y_1 + x_0y_2] + 2[x_1y_0 + x_0y_1] + x_0y_0 \end{aligned} \quad (3.6)$$

Presenting the calculation in this form clarifies an approach to its hardware implementation. Create an accumulator register with a left-shift function. First accumulate the product x_3y_3 . Then perform a left-shift operation, and accumulate x_3y_2 and x_2y_3 . Continue until we reach x_0y_0 . By this time, the first product will have left-shifted six times, effectively multiplying it by 64 as intended.

To accomplish this, operand bits must be accessed from memory in a “zig-zag” pattern, shown in figure 3.2. In this pattern, each diagonal is traversed accumulating one-bit products. After each diagonal is complete, the accumulator should be left-shifted before the next diagonal begins accumulating.

Implementing this in hardware requires a special “zig-zag” address generation unit (AGU) which generates the address in this unusual pattern. These addresses are used to retrieve the required data from memory. An implementation of this technique is shown in figure 3.3.

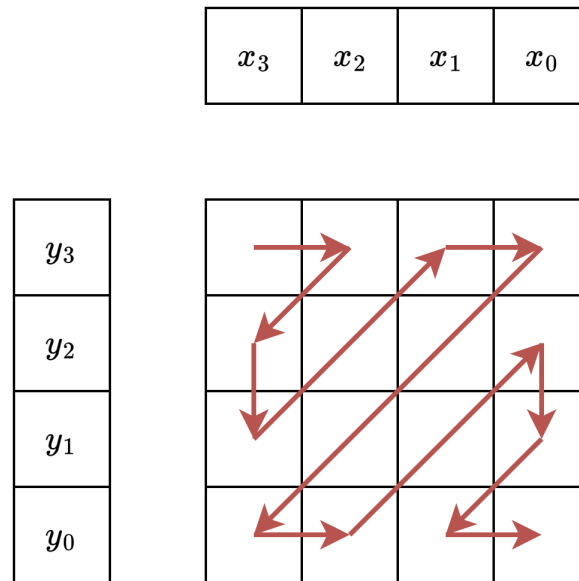


Figure 3.2: “Zig-zag” memory access pattern. Each cell represents a one-bit binary product with one bit from x and one bit from y .

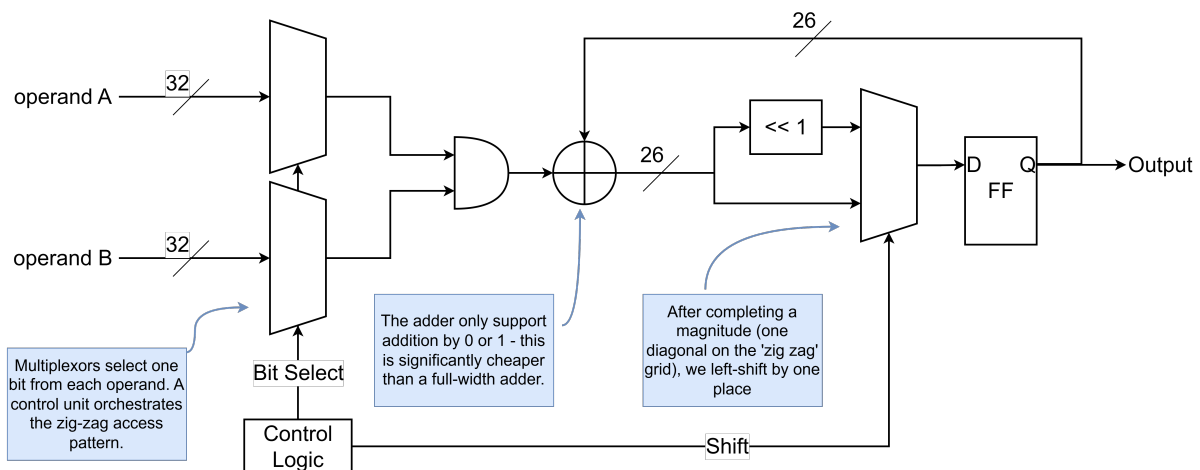


Figure 3.3: Simple hardware to implement the bit-serial multiplication technique described above. The control logic generates the zig-zag access pattern, which could be shared over several parallel processing units. The products are accumulated in an accumulator register with a left-shift function.

The idea presented here can be further extended to sum the operands of several multiplication (such as a vector-vector product) in a single pass, by simply accumulating the products of each multiplication together.

The bit-serial arithmetic unit is presented in [4] is capable of performing bit-serial matrix-vector multiplications. It is referred to herein as a Matrix-Vector Unit (MVU). It operates on a vector of 64 elements (each at one bit of precision) and a 64×64 matrix (again with each element only one bit). The output vector is fed into an accumulator which allows aggregating for bit-serial calculations.

In this work, the architecture presented in [4] is modified and then integrated into a larger system. The remainder of this section details the changes made and their impact on the final system.

3.1.4 Transposed Memory Format

Bit-serial operation requires that individual bits of operands be retrieved from memory one-by-one, and individual bits of the result stored one-by-one. There are two naïve approaches to retrieving individual bits from operands for bit:

- Extract the whole operand word from memory and multiplex the appropriate bit. However, memory accesses are a critical bottleneck to the performance of NN accelerators, and this approach drastically increases the rate of memory transfers.
- Extract the whole operand into a shift register, and shift the bits out one-by-one. However this will require a large number of shift registers to vector-vector multiplication which has many operands.

A more intelligent approach is proposed in [4] where operands are stored in a transposed format in memory. Each addressable 64-bit word in memory represents one bit from 64 different values. After a calculation has accumulated its final result, it is fed back into

memory one bit at a time using a shift register. This is discussed in greater detail in the following section.

3.2 Existing MVU Design

[4] presents a matrix-vector unit (MVU) design which implements bit-serial arithmetic units to accommodate variable precision calculations for neural network inference.

The original design is composed of several key components:

- An APB interface for control and configuration of the processing elements
- Separate internal memory banks for storing data, weights, scalers, and biases
- A simple external interface for reading/writing the internal memory banks
- An array of MVU instances, each with:
 - Configurable AGUs for generating addresses to read from internal memory banks. There are separate AGUs for input data, weights, scalers, biases, and output data.
 - A datapath for performing matrix-vector multiplication, scaling, bias addition, max-pooling/ReLU, and quantisation/serialisation.

3.2.1 High-Level Operation and Registers

The MVU provides control-and-status registers which are written by the core to prepare a calculation and trigger its execution. The control registers include:

- Pointers to the base address for input data, weights, scalers, and output data.
- Jump schedules (the lengths and strides of nested for loops) for each AGU (input data, weights, scalers, and output data).
- Precision settings for the input data, weights, and output data.

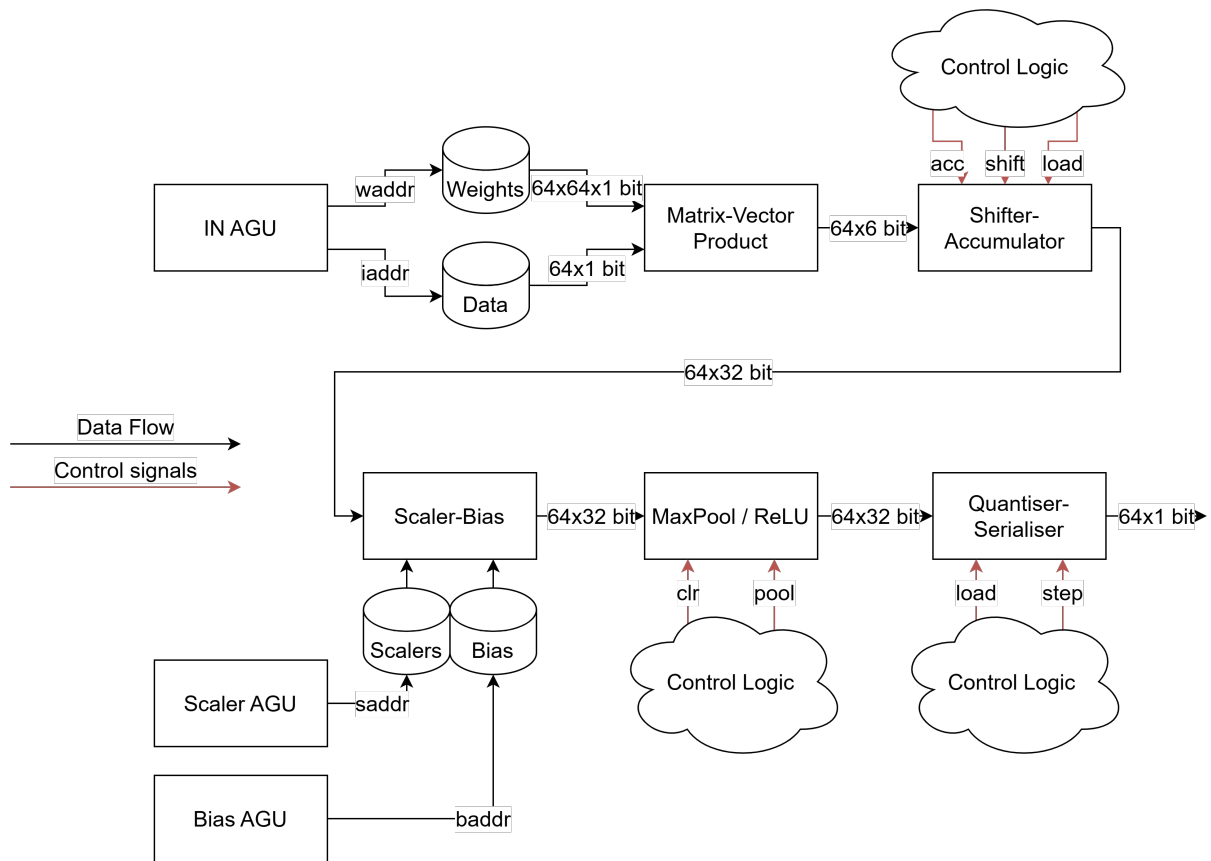


Figure 3.4: The datapath of the existing MVU design presented in [4].

- A timer value to count down until an operation is complete.
- A status register indicating when operation is complete.

There is a special ‘command’ register which, when written to, triggers the operation to begin execution. Once the operation has complete, an interrupt request (IRQ) is raised to signal to the core that the operation is complete and result is ready to be read from the MVUs memory.

3.2.2 Datapath

The datapath is the core of the design, and is shown in greater detail in figure 3.4.

An ‘input AGU’ (which includes the zig-zag AGU) generates addresses to read input data

and weights from internal memory banks. The input data is a vector of 64 bits, multiplied by a weights matrix of 64×64 bits. The result is an array of 64 6-bit words, which are fed into an accumulator to accumulate over several bits in bit-serial fashion (see section 3.1). The result is fed into a scaler/bias unit which can apply fixed scaling and add a bias from memory. The Maxpool/ReLU unit allows applying a max-pooling or ReLU activation function. Finally, the quantiser/serialiser unit can quantise the result to a lower bit-width and serialise it for writing back to memory in bit-transposed format. The serialisation will take several clock cycles (outputting one bit per cycle), during which the next calculation can begin accumulating in the accumulator.

3.2.3 Data Banks

Each MVU instance has several internal memory banks.

Data type	Capacity	Addressable as...	Internal Access	External Access
Input/output data	2 Mb	32k lines of 64 bits	Read/Write	Read/Write
Weights	2 Mb	512 lines of 4,096 bits	Read-only	Write-only
Scalers	64 kb	64 words of 1,024 bits	Read-only	Write-only
Biases	128 kb	64 words of 2,048 bits	Read-only	Write-only

Table 3.1: Overview of the internal memory banks of each MVU instance in the original design.

For implementation on Xilinx FPGA, the internal memory banks are implemented using block RAM (BRAM) primitives. These primitives allow users to specify concretely that FPGA's BRAM resources should be used to implement the memory, rather than the less-efficient approach of implementing memory using registers and logic. The BRAM primitives also support having two access ports to allow simultaneous access – for example, allowing the MVU to read input data for calculation while simultaneously writing the result of a previous calculation back to memory.

Part II

Work Completed

Chapter 4

Processing Element

4.1 Modifications to the MVU

4.1.1 Bug fix: Data banks 1-31 unusable

In the original design, each matrix-vector unit had 32 banks for storing data, each with 1,024 addressable lines of 64 bits. Banks were numbered from 0 to 31.

The data banks have three possible access methods:

- A read/write port allows the MVU to read and write data stored locally (“d” port, for “data”)
- A read/write port allows an external interface to access data inside the MVU’s (“c” port, for “controller”)
- A write-only port allows MVU’s to write to each other’s data banks via an interconnect (“i” port, for “interconnect”)

The designers intended it be possible for two ports to simultaneously read or write provided they were accessing different banks. However, the deconfliction logic is fundamentally flawed. A diagram of the original design is shown in figure 4.1, with only two data banks

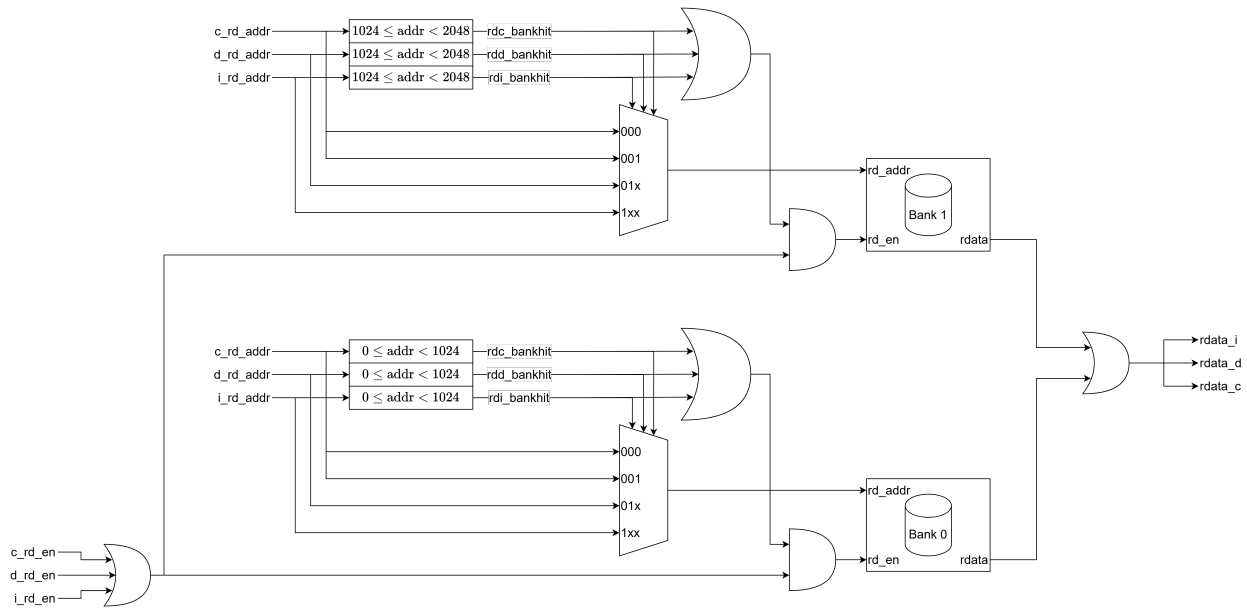


Figure 4.1: The original RTL design of the data banks in the MVU. There is a critical bug described below.

(it may be generalised to the full 32-bank design). The design assumes that when the `rd_en` signals is low on a memory block, the output data will read as zeros.

The design provides read logic for the interconnect, however the interconnect does not facilitate MVU's *reading from* one another's data banks - only *writing to*. Accordingly, the interconnect read address has been tied to zero (elsewhere in the design). The result is that the signal `rdi_bankhit` on bank 0 is always high. As a result, if the data or controller ports attempt to read from a non-zero bank, both banks will be returning data, which will be OR'd together in the final stage. The same issue also applies the data and controller ports are both attempting to read from different banks.

The tests originally written for this hardware only access data addresses below 1024, and do not use the controller bus. This may be the reason this bug was not previously identified.

The logic for accessing data banks was reworked to make all banks accessible for one write and one read on any clock cycle.

4.1.2 Bug fix: Race conditions in sequential logic designs

A number of components of the original design made use of Verilog blocking assignments to describe sequential logic (shown in listing 4.1). The correct design pattern involves non-blocking assignments (shown in listing 4.2).

```

1 always @(posedge clk) begin
2     if(cclr) begin
3         0 = 0;
4     end else if(clk) begin
5         if (load) begin
6             0 = I;
7         end else if (acc) begin
8             if(sh) 0 = 0+0+I; /* Shift left of accumulator */
9             else    0 = 0+I;   /* Plain accumulate */
10        end
11    end
12 end

```

Listing 4.1: The original implementation of the shift-accumulate functionality, with incorrect usage of Verilog blocking assignments.

```

1 always @(posedge clk) begin
2     if(cclr) begin
3         0 <= 0;
4     end else begin
5         if (load) begin
6             0 <= I;
7         end else if (acc) begin
8             if(sh) 0 <= (0<<1)+I; /* Shift left of accumulator */
9             else    0 <= 0+I;     /* Plain accumulate */
10        end else begin
11            0 <= 0; // hold value
12        end
13    end
14 end

```

Listing 4.2: The corrected implementation of the shift-accumulate functionality.

The original implementation yielded inconsistent waveforms for pre-synthesis and post-synthesis simulations, which is extremely hazardous for verification. The fixed implementation is the standard design pattern. Note that there have also been changes to code style to improve readability.

Similar issues were identified in the quantiser-serialiser module, top-level controller module, and the zig-zag AGU.

4.1.3 Register Map Redesign

The original implementation employed an APB interface to control a set of latches.

APB (Advanced Peripheral Bus) is a standard published by Arm for controlling peripheral devices. While straightforward to implement, its operation is slow, with a minimum of two clock cycles per read/write transfer. In this project, the bus was upgraded to AHB (Advanced High-performance Bus), a more advanced Arm standard with support for pipelined operation to improve throughput.

The original register map implementation employed latches with undefined reset values for each configuration register. While functional, latches are considered non-standard and undesirable. An excerpt of the original code is shown in listing 4.3.

```

1 MVU_CFG_INTERFACE mvu_cfg_if(); // Create the MVU config interface
2
3 logic [mvu_pkg::BMVUA-1:0] mvu_id;
4 assign mvu_id = register_adr[APB_ADDR_WIDTH-1:12];
5     // Select the MSBs of the address as the MVU ID for this transaction
6
7 // Write logic for APB interface
8 always_comb begin
9     // APB register write logic
10    if (apb_write) begin
11        unique case (mvu_pkg::mvu_csr_t'(register_adr[11:0]))
12            mvu_pkg::CSR_MVUWBASEPTR : mvu_cfg_if.wbaseaddr[mvu_id] = apb.
                pwdata[BBWADDR-1 : 0];
13            mvu_pkg::CSR_MVUIBASEPTR : mvu_cfg_if.ibaseaddr[mvu_id] = apb.
                pwdata[BBDADDR-1 : 0];
14            /* ... further register omitted for brevity ... */
15        endcase
16    end
17 end

```

Listing 4.3: An extract from the original implementation of the register map for MVU control registers. Note the use of an `always_comb` which are intended to create clouds of combinational logic. Registers cannot be implemented using purely combinational logic. This strongly suggests the use of inferred latches was unintentional by the original developers. Flip flops (using `always_ff`) are the preferred design pattern in modern implementations.

Additionally, there were a large number of configuration registers whose outputs did not drive any logic and appeared to have no use. These have been removed.

Finally, the revamped register map included a secondary set of “shadow register”. Data

is initially written to the shadow registers, and the “live” registers only updated when a new operation is triggered. This extension allows configuration registers to be written while a previous one is still completing, without corrupting the ongoing operation. This change avoids a ‘transition’ period where, after an operation completes, there is idle time while the next is being setup. Instead, the setup for the next operation can be anticipated, and ‘triggered’ when the previous finishes. This is vital to ensure efficient utilisation of the hardware resources.

4.1.4 Memory Interface Redesign and AHB Integration

In addition to rewriting the control registers to interface via AHB, the internal memories of the MVU saw improvements to their external interfaces.

There are four categories of internal memory in the MVU: data, weights, scalers, biases. The latter three can only be read by the MVU, while the former may be written or read. They vary in word size:

- The MVU accesses **data** in lines of size 64 bits. This value was chosen by the original designers as convenient for many neural network architectures – the number of channels in a convolution neural network layer is typically a multiple of 64.
- To allow any channel in a layer to contribute to any channel in the next layer, the **weights** are stored in lines of 4,096 bits (interpreted by the processing element as a 64×64 matrix).
- **Scalers** for each individual channels are 16 bits - allowing for 64 channels, scalers are read from memory in lines of 1,024 bits.
- **Bias** values are stored as 32 bits and therefore read in lines of 2,048 bits.

These very large word sizes are efficient for the internal memory access patterns of the MVU. However, they are problematically large for interfacing with a system bus, which

typically employ word sizes from 8-64 bits. For this project, a 32-bit AHB was used to integrate the MVU with a broader system.

Initial approach: 2-port RAM with different word sizes To connect the 32-bit data bus with the larger MVU word sizes, two options were considered initially:

- Implement a wrapper around the MVU which translates a single 32-bit write into a read-modify-write cycle. This would triple the latency in each transaction. To avoid this overhead, an improvement was conceived called “lazy-writeback”, wherein the modified word is only written back to the MVU’s data when a new transaction seeks to access a *different* line. This would eliminate large numbers of read/write transactions, at the expense of more complex state machine logic in the wrapper.
- Modify the memory banks inside the MVU to provide two ports of distinct widths: a 32-bit port for the external interface, and a wider port for internal memory access.

The former strategy required less modifications to the internal code of the MVU’s. However, the increased complexity and transaction latency was deemed not to be worthwhile, so the latter strategy was chosen. A parametrisable SystemVerilog module was written named `ram_2port`. A high-level diagram is shown in figure 4.2 for an example with a read-word size of 128.

The module provides a write port and a read port, each addressable. The write address space is larger and operates on 32-bit words, while the read address space is smaller and acts on larger words (or “lines”). The most significant bits of a write address select a line following the same memory map as the read port, while the less significant bits select a 32-bit word within the line. A SystemVerilog ‘`generate for`’ construct is employed to generate the correct number of RAM blocks.

A standalone Vivado synthesis flow was run on the new module with a variety of configurations to ensure Vivado correctly inferred block RAM rather than rely on flip flops. The SystemVerilog description is shown in listing 4.4.

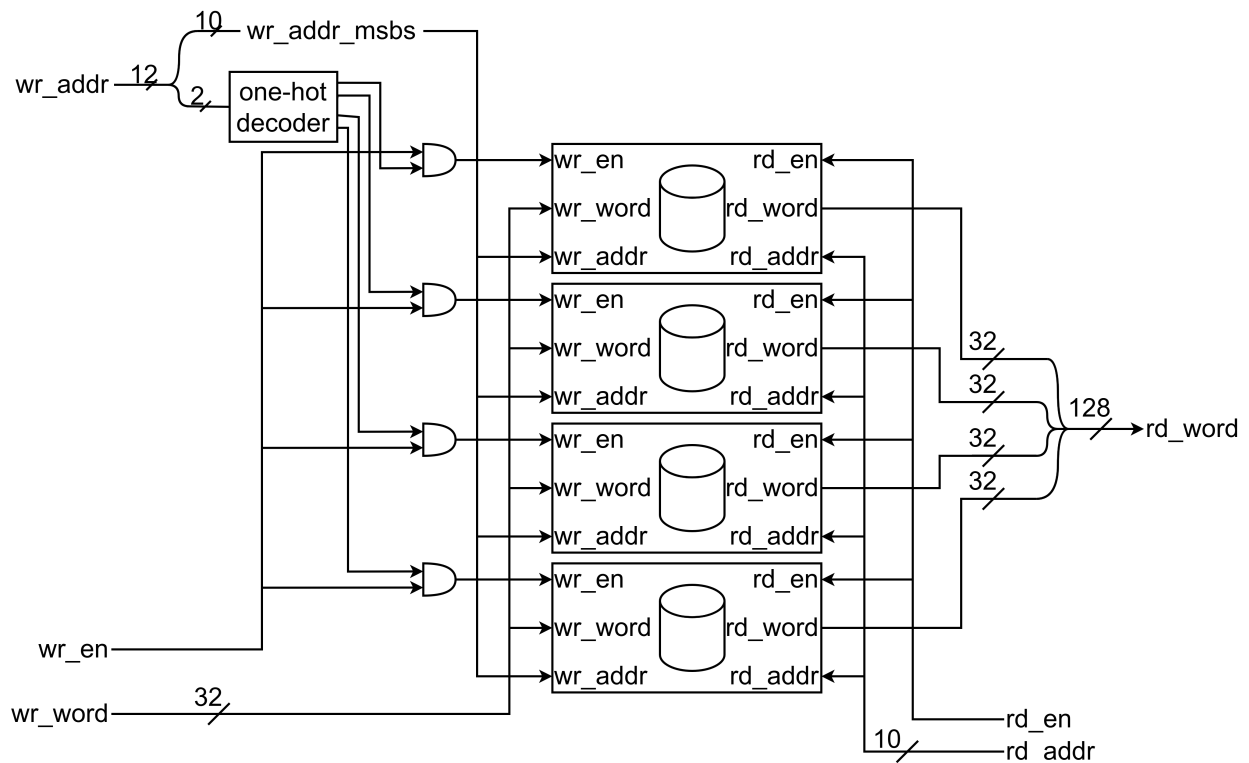


Figure 4.2: An RTL diagram of a design for a 2-port RAM module. The write port (on the left) uses 32-bit words and 12-bit address, while the read port (right) uses 128-bit words and 10-bit addresses. All signals are synchronous to a shared clock which is not shown for readability.

```

1 module ram_2port #(
2     parameter WR_WORD = 32, // Size of the words on the write port (in
      bits)
3     parameter RD_ADDR = 10, // Size of the address on the read port (size
      of the write port address will be inferred from this and the ratio)
4     parameter RD_WORD = 256, // Number of bits read at a time
5     localparam RD_RATIO = RD_WORD / WR_WORD, // Ratio of the read word
      size to the write word size (e.g. for 128-bit read words and a 32-
      bit write word, this should be 4. $clog2(RD_RATIO) bits will be
      added to the read address width to get the write address width,
      with the LSB's used for sub-words)
6     localparam BLOCK_SEL_BITS = $clog2(RD_RATIO),
7     localparam WR_ADDR = RD_ADDR + BLOCK_SEL_BITS // Address space for
      writes is larger because we write smaller words
8 ) (
9     input  wire          clk,
10    input  wire          rd_en,
11    input  wire [RD_ADDR-1 : 0] rd_addr,
12    output logic [RD_WORD-1 : 0] rd_word,
13    input  wire          wr_en,
14    input  wire [WR_ADDR-1 : 0] wr_addr,
15    input  wire [WR_WORD-1 : 0] wr_word
16 );
17 // If you require a different word size for writing vs. reading (e.g.
18 // write 32 bits for standard system bus,
19 // read 128-bits to feed a wide interface), you can use this module to
20 // "reshape" your memory.
21 // The read address space is specified by RD_ADDR (e.g. 10 bits for
22 // 1024 addresses)
23 // The write address space has LSB's appended to index individual
24 // words in that address space, so it is larger by $clog2(RD_RATIO)
25 // bits
26
27 logic [BLOCK_SEL_BITS-1:0] wr_block_select;
28 logic [RD_ADDR-1:0] wr_addr_msbs;
29 assign wr_block_select = wr_addr[BLOCK_SEL_BITS-1:0];
30 assign wr_addr_msbs = wr_addr[WR_ADDR-1 : BLOCK_SEL_BITS];
31
32 genvar i;
33 generate for (i=0; i<RD_RATIO; i=i+1) begin : gen_rams
34     logic [WR_WORD-1 : 0] mem [(2**RD_ADDR)-1:0];
35     always @(posedge clk) begin
36         if (rd_en) begin
37             rd_word[i*WR_WORD +: WR_WORD] <= mem[rd_addr];
38         end
39         if (wr_en && (wr_block_select == i)) begin
40             mem[wr_addr_msbs] <= wr_word;
41         end
42     end
43 end generate
44 endmodule

```

Listing 4.4: SystemVerilog implementation of a 2-port RAM with different word-sizes for

the read and write ports. Vivado will correctly infer block RAM's rather than relying on flip-flops.

Unfortunately, after considerable effort and analysis to implement this technique, it was found that although Vivado correctly infers block RAMs when the design is synthesised in isolation, it does not do so when the module is integrated into the full design. This could be addressed with an out-of-context synthesis flow, but it seemed that it would be appropriate to avoid relying on Vivado to correctly infer block RAMs and instead explicitly instantiate Xilinx block RAM primitives. This approach had been previously avoided as a block RAM primitive does not allow a 32-bit write port on the same memory as a 4096-bit read port, which is necessary for the weights memory. However, it was later realised that a similar effect could be accomplished using byte-enable signals.

Second approach: Byte-enable signals Rather than implementing a RAM with two ports of different widths, RAM blocks were created with a byte-enable signal on the write port, which allows writing only to specific bytes in the selected line of memory.

The Vivado Block Memory Generator was used to instantiate block RAM primitives with two ports: one for the controller to write, one for the MVU to read.

This means it is possible to write entire lines of memory in a single clock cycle, but the byte-enable signals can also be used to write only 32 bits at a time, which is necessary to integrate with a 32-bit AHB. This approach is much simpler than the previous one, and does not rely on Vivado to infer block RAMs correctly.

The MVU AHB wrapper was modified to set the appropriate byte-enable bits based on the least significant bits of the address being written to, and to multiplex the appropriate 32 bits from any read data based on the same. Effectively, the memory is addressed as 32-bit words from the perspective of the AHB.

4.2 Verification

In order to verify the behaviour of the MVU, a Python model was created to simulate the behaviour of the MVU at a high level.

The model could be supplied a NumPy array representing the input feature map (of form $\text{Width} \times \text{Height} \times \text{Channels}$) and a second array representing a kernel to perform convolution with (of form $\text{Kernel Width} \times \text{Kernel Height} \times \text{Input Channels} \times \text{Output Channels}$). The model used NumPy's standard functionality to perform convolution, then quantised the output data to the appropriate number of bits.

The Python suite also included tools for randomly generating feature maps and kernels, and dumping these to the transposed format expected by the MVU.

This facilitated efficient testing of the MVU design by randomly generating convolution data and kernels, simulating the design with the data, and comparing the output to the expected output from the Python model. A number of bugs were identified and correctly, largely concerning the specific detail of memory layout (e.g. $\text{Height} \times \text{Width}$ vs. $\text{Width} \times$

Height).

Chapter 5

System Integration

5.1 SoC Architecture

The MVU is only capable of perform multiplication operations in response to an instruction, under various configurations. It does not provide high-level flow control, input-output management, or other features necessary to implement a useful NN-enabled system.

For this reason, it is integrated into an SoC which provides this functionality. The integration process also serves as a route to improved verification and evaluation, usability testing, and proof-of-concept for a sophisticated use case.

The SoC is named Dynamic Neural Network System-on-Chip or “**DyNNSoC**”.

Such an SoC is generally comprised of a CPU core (or controller), system bus(es) for communicating between components, and peripheral components. What follows is a description of the components included in this project. A high-level block diagram of the SoC is shown in Figure 5.1.

Each component is described in brief below, with select components being described in more detail in later sections

- **Ibex (core)**: See subsection 5.1.2.
- **MVU Array**: See subsection 5.1.1.

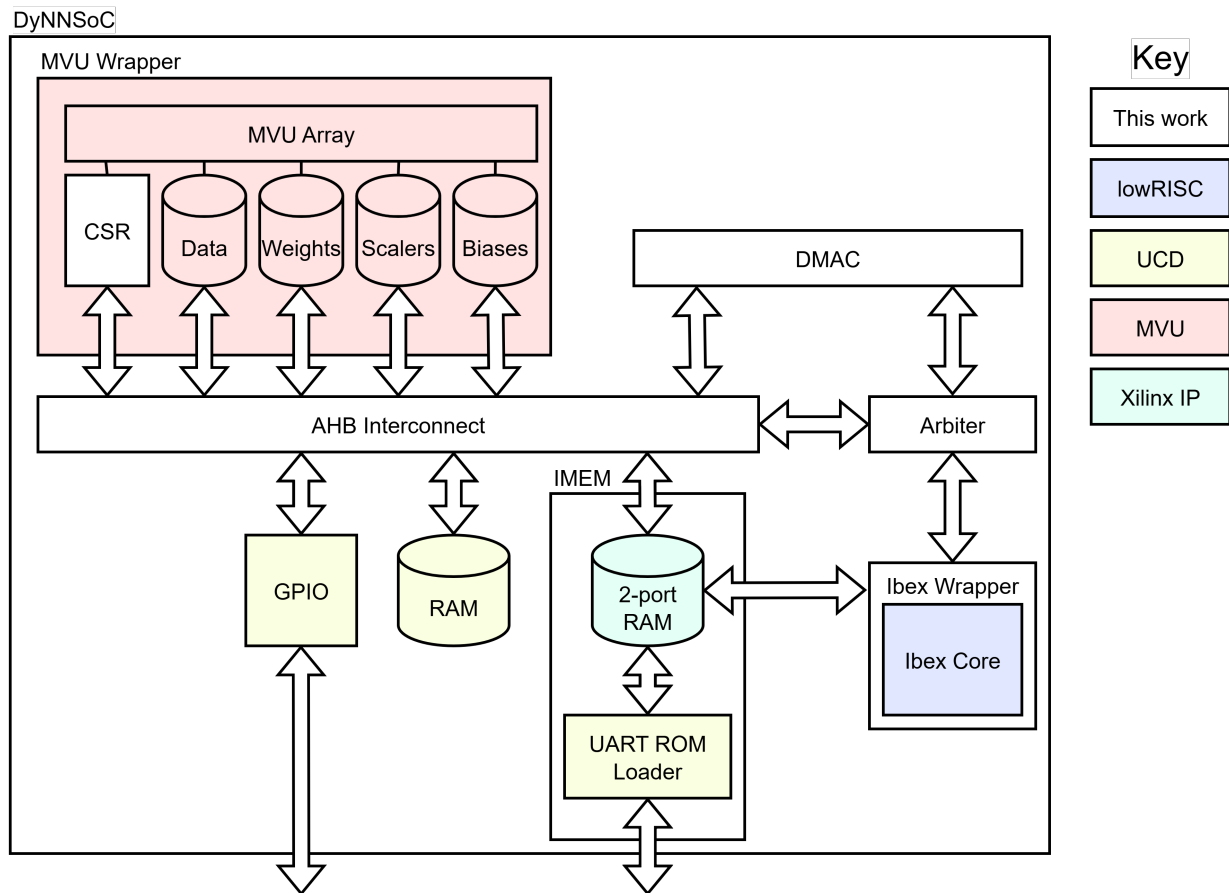


Figure 5.1: High-level block-diagram of the SoC, showing the core, peripherals, and AHB infrastructure. In blue is the Ibex core from lowRISC [16]. Content in green is Xilinx FPGA primitives. Content in yellow is taken from module content for Digital & Embedded Systems (EEEN40280). Content in white was design for this project. Content in red is from [4], though with substantial modifications.

- **Instruction Memory (‘IMEM’)**: See subsection 5.1.6.
- **DMAC**: See subsection 5.5.1.
- **AHB Arbiter**: See section 5.6.
- **RAM**: General-purpose read/write memory. See subsection 5.1.3.
- **GPIO**: General-purpose input/output peripheral. See subsection 5.1.4.

This selection is a minimal example of the components necessary for a useful SoC. However, the use of a standard, common system bus allows other components to be easily added.

5.1.1 MVU Array

The MVU processing element as described in section 4.1 is integrated into the SoC. The number of MVU cores is configurable – a value of 4 was chosen for this project as it is the maximum that could fit onto the FPGA being used, but this can be set as large as 16 without issue.

5.1.2 Ibex: RISC-V SoC Core

The lowRISC *Ibex* core was chosen for this project [16]. It implements the RISC-V instruction set. As an open standard, RISC-V is widely supported and has a large ecosystem of tools and software.

Ibex is a small, open-source core. It employs a 3-stage pipeline to fetch, decode, and execute each instruction. It supports the “m” extension for RISC-V which enables multiplication and division instructions in hardware.

Ibex does not have an AHB-compliant interface, instead exposing two simple memory interfaces: one for accessing RAM, and one for retrieving instructions.

The work in [5] created a wrapper around the Ibex core which translates these native memory access protocols to conform to the AHB standard, unifying them into a shared AHB interface. This wrapper was used in previous iterations of this project to integrate Ibex.

However, retrieving RISC-V instructions over the same bus as data created a performance bottleneck. The Ibex core must fetch new instructions on (almost) every clock cycle, keeping the AHB busy at all times. Any load/store instructions will interrupt the instruction fetches, and any lower priority AHB transactions will be delayed indefinitely.

To address this, this work created a new wrapper which connects the instruction fetch interface of the Ibex core directly to the instruction memory (ROM) via a simpler side-channel memory interface, while the AHB is used exclusively for data access. This allows data transactions to occur without interrupting instruction fetches, and allows the AHB to be shared between multiple components without creating bottlenecks. This interface is visible between the instruction memory and Ibex core in the block diagram in figure 5.1.

5.1.3 RAM

This design was taken from the Digital & Embedded Systems (EEEN40280) module content with minimal modifications. It provides general-purpose read/write memory for the SoC, used by the C runtime for storing the stack and heap.

The module implements an AHB interface and connects it to a block of Xilinx block RAM IP. It also handles read-after-write conflicts where a read that takes place too soon after a write to the same address can cause incorrect data to be read back.

5.1.4 GPIO

A general-purpose input/output (GPIO) peripheral. The design was taken from module content for EEEN40280 Digital & Embedded Systems.

This provides

- two read-only registers which return data from input pins exposed by the SoC.
- two read-write registers which show the data written to them on output pins exposed by the SoC

This is used by testbenches to interact with the SoC in a simple fashion.

- Input pins can be used as a source of entropy for randomised testing
- Special output codes 0x900D ('good'), 0x0BAD ('bad'), and 0xBEEF are used by firmware to mark passing tests, failing tests, and the end of the test suite.
- Output can also be used to indicate the progress of a test case, which is useful for debugging.

5.1.5 ROM Loader

To be able to write to the instruction memory to initialise new firmware, A ROM loader module is borrowed from the Digital & Embedded Systems (EEEN40280) module content. This receives data over a serial UART connection and passes it to a simple write-only interface for writing data to the instruction memory.

5.1.6 Instruction Memory

The Ibex wrapped provides an instruction fetch interface for retrieving instructions from memory.

It is also necessary to be able to access instruction memory via the AHB, as many compilers will include read-only program data and initialised variables with the program data, meaning these must be accessible as *data*, not as instructions.

Finally, a write-only interface must be provided to allow the instruction memory to be initialised with new firmware.

The signature of the instruction memory module is shown in Listing 5.1, showing these three interfaces.

```

1 module imem (
2     input wire HCLK,           // bus clock
3     input wire HRESETn,       // bus reset, active low
4
5     // AHB slave interface - read only
6     ahb_intf_s.slave AHB_IF,
7
8     // Memory interface for core instruction fetches - also read only
9     instruction_fetch_if.rom instr_if,
10
11    // UART loader connections
12    input wire resetHW, // hardware reset
13    input wire loadButton, // pushbutton to activate loader
14    input wire serialRx, // serial input
15    output [11:0] rom_load_status, // 12-bit output to indicate
        progress
16    output rom_load_active // loader active
17);
18 /*
19  The instruction memory instantiates three modules:
20      - A 2-port RAM
21      - A RX-only UART
22      - A "RAM loader" which interprets the incoming UART data and
        writes to RAM
23  */
24 endmodule

```

Listing 5.1: SystemVerilog module signature of the instruction memory store

The RAM blocks provided on Xilinx FPGAs only allow two interface. Fortunately, it can be assumed that the core is not operating while the memory is being initialised, so the write-only interface can be multiplexed with the instruction fetch interface.

5.2 EDA Workflow

DyNNSoC was developed with a complex EDA workflow involving multiple languages and tools. Regression testing provided continuous confidence in the design as it evolved. The tools that were used are explained below. All non-autogenerated files were version controlled in Git and the open-source repository is available at [18].

5.2.1 Languages, Compilers, and Simulators

RTL DyNNSoC’s hardware was developed primarily in SystemVerilog, with some pre-existing components in Verilog, and Xilinx IP described in IPXACT.

Vivado was used for simulation, synthesis, implementation, and power analysis.

Firmware Firmware was developed in C and compiled using the GNU toolchain for RISC-V [21]. The RISC-V `objcopy` utility allowed generating the compiled code in hex format, then Vivado’s `$readmemh` construct was used to load the hex into the SoC’s instruction memory during simulation.

A central `firmware.mk` file was used to manage this build process. It is a Makefile, used by the Make build system. It provided the standard C header files for the SoC (with macros defined for memory-mapped registers and interrupt numbers) and a C runtime. The C runtime and linker script ensure that initialised variables are copied from ROM into RAM at program startup, so they can be modified while the firmware is running. It also creates symbolic names for the interrupt handlers at the correct addresses in ROM, allowing the firmware to define the handlers as needed. This Makefile was included as a dependency in more complex Makefiles for individual test cases and build targets.

Three source files are included in all firmware builds: `crt0.S`, `dynnsoc.h`, and `dynnsoc.c`. `crt0.S` is the assembly startup code which sets up the initial execution environment before calling `main()`. This includes setting up the stack pointer, copying initialised variables from ROM to RAM, and jumping to the main function. `dynnsoc.h` is a header file containing SoC-specific definitions, register mappings, and utility macros for firmware development. `dynnsoc.c` contains useful API methods for interacting with the SoC such as reading and writing data from the MVU in the transposed format described previously.

These sources files, and the main source file for a particular test case, are compiled into object files in the usual fashion. The object files are then linked together to create an ELF binary using the GNU RISC-V linker. The linker has a configuration script `link.ld` which

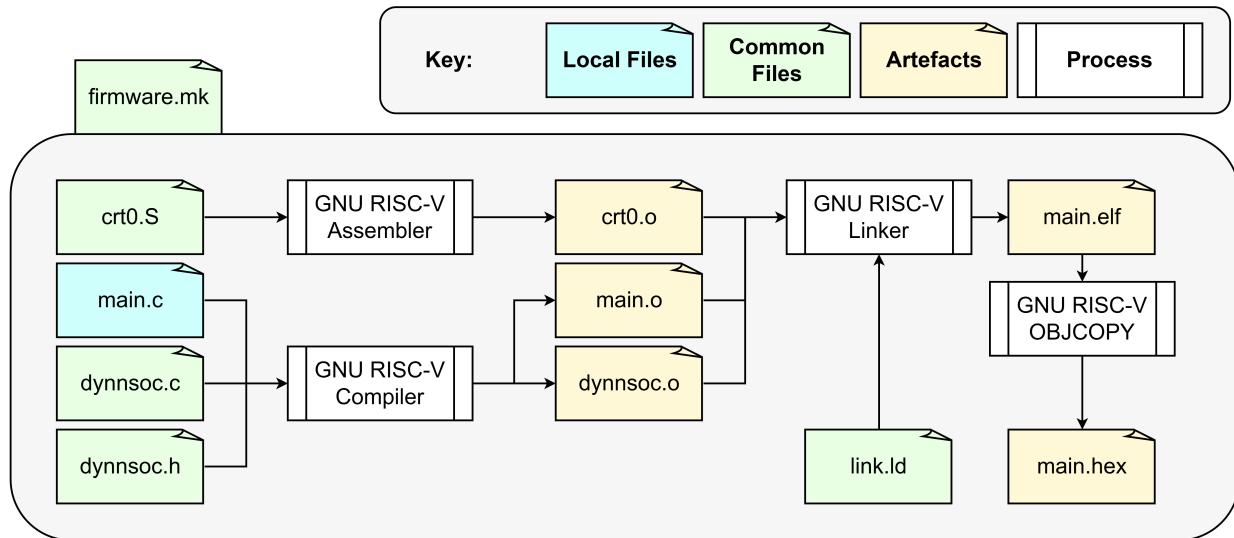


Figure 5.2: The process set out by `firmware.mk` for building firmware for DyNNSoC. `firmware.mk` is a Makefile which provides a common flow for building firmware. It can be included as a dependency in more complex Makefiles for test runs where it is invoked before each run.

specifies appropriate memory layout for the SoC including locations for the stack and heap, and ensures data is word-aligned.

The ELF file is then converted into a Verilog hex file using the GNU `objcopy` utility with the `-O verilog --verilog-data-width 4` flags.

5.2.2 Use of FuseSoC

FuseSoC [12] is a command-line application which provides a build system for EDA projects, with a focus on enabling core re-use. DyNNSoC is described by a FuseSoC `.core` file, a portion of which is shown in listing 5.2.

```

1 CAPI=2:
2 name: thomasodea:dynnsoc:dynnsoc:0.1
3 description: SoC with accelerator for dynamically-quantised neural
   networks
4
5 # This is a FuseSoC "core" file
6 # It provides the necessary information for building, simulating, and
   synthesising the SoC design
7 # It has dependencies on other core files which must be included in your
   FuseSoC library

```

```

8
9 filesets:
10     rtl_base: # base file list - includes everything except the MVU
11         file_type: systemVerilogSource
12         files:
13             - rtl/slaves/imem.sv
14             # full file list has been omitted for readability
15             - rtl/top/dynnsoc.sv
16
17         depend: # include the Ibex core
18             - lowrisc:ibex:ibex_core:0.1
19
20     # We may choose to include "rtl_full" or "rtl_lite" in addition to "
21     # rtl_base"
22     # this is useful for excluding the MVU when we are just testing the
23     # rest, since the MVU is very large and slows down simulation
24     # significantly
25     rtl_full:
26         file_type: systemVerilogSource
27         files:
28             - rtl/top/mvutop_wrapper.sv
29         depend:
30             - thomasodea:mvu:mvu:0.1
31
32     rtl_lite:
33         file_type: systemVerilogSource
34         files:
35             - rtl/stubs/mvutop_wrapper_STUB.sv
36
37     # Constraint file for synthesis with Vivado
38     constraints:
39         file_type: xdc
40         files:
41             - dynnsoc.xdc
42
43 targets:
44     impl:
45         filesets:
46             - rtl_base
47             - rtl_full
48             - constraints
49         toplevel: dynnsoc
50
51     default_tool: vivado
52     tools:
53         vivado:
54             part: xczu9eg-ffvb1156-2-e

```

Listing 5.2: The .core file used to describe DyNNSoC in FuseSoC

The .core file allows various flows to be invoked conveniently from the command line including synthesis, implementation, various test cases, and power analysis, with commands

such as:

```
1 fusesoc run --target=synth dynnsoc
```

5.2.3 Use of MVU Stub Files

The MVU represents the majority of the logic in DyNNSoC and its inclusion significantly increases the time taken for elaboration, synthesis, and implementation. This delay was extremely cumbersome when performing integration tests and debugging the remainder of the SoC.

To address this, an MVU ‘stub’ was created which provides the same interface as the MVU but with no internal logic. Inputs are ignored and output signals are driven to sensible defaults. An alternate file list (visible in listing 5.2) was defined which used this stub, and this file list was used for test cases which did not rely on the MVU. This technique allowed far faster, smoother integration testing.

5.3 AHB Infrastructure

The Advanced High-performance Bus (AHB) is a widely-used bus protocol in SoC design, standardised by Arm [1]. It provides an *addressable* bus with a master/slave distinction (referred to in newer texts as manager/subordinate), meaning a master component initiates transactions which either read or write data to an *address* in a slave component.

5.3.1 SystemVerilog Interface

For this project, a variety of AHB infrastructural components were created to enable easy integration of new components into the SoC. Listing 5.3 shows an example of the `ahb_intf_s` SystemVerilog interface, which defines the signals for connecting an AHB slave to the interconnect. There is also a matching master-interconnect interface. The `modport` construct

enforces the direction of signals to avoid erroneously driving signals in the wrong direction. This interface (and similar constructs) enables rapid, flexible SoC development.

```

1 interface ahb_intf_s #( // Signals for connecting a slave to the AHB
2     interconnect
3     parameter ADDR_WIDTH = 32,
4     parameter DATA_WIDTH = 32
5 );
6     // Slave drives these
7     logic [DATA_WIDTH-1:0] HRDATA;
8     logic HREADYOUT;
9     logic HRESP;
10
11    // Interconnect/master drives these
12    logic [DATA_WIDTH-1:0] HWDATA;
13    logic HWRITE;
14    logic [2:0] HSIZE;
15    logic [3:0] HPROT;
16    logic [1:0] HTRANS;
17    logic [ADDR_WIDTH-1:0] HADDR;
18    logic HREADY;
19    logic HSEL;
20
21    modport slave (
22        input HSEL, HREADY, // from interconnect
23        input HWDATA, HWRITE, HSIZE, HPROT, HTRANS, HADDR, // from master
24        output HRDATA, HREADYOUT, HRESP
25    );
26
27    modport interconn (
28        output HSEL, HREADY,
29        output HWDATA, HWRITE, HSIZE, HPROT, HTRANS, HADDR,
30        input HRDATA, HREADYOUT, HRESP
31    );
32 endinterface

```

Listing 5.3: SystemVerilog defining a standard ‘interface’ (collection of signal) for connect slave components to the interconnect.

5.3.2 AHB Interconnect

A single ‘interconn’ module is instantiated at the SoC top-level. Listing 5.4 shows its straightforward connection to each slave.

```

1 ahb_interconn interconn (
2     .HCLK,
3     .HRESETn,
4     .master_if    ( ahb_arbitrated_if.interconn ),

```

```

5  .slave_if_s0 ( ahb_imem_if.interconn      ),
6  .slave_if_s1 ( ahb_ram_if.interconn      ),
7  .slave_if_s2 ( ahb_gpio_if.interconn     ),
8  .slave_if_s3 ( ahb_uart_if.interconn     ),
9  .slave_if_s4 ( ahb_mvu_if.interconn      ),
10 .slave_if_s5 ( ahb_dmac_s_if.interconn    )
11 );

```

Listing 5.4: SystemVerilog showing the interconnect being connected to each slave.

This module provides memory-mapping as explained in section 5.4. This is the only system-level modification necessary to integrate new components.

The design for the interconnect was adapted from standard Verilog provided by Arm.

5.3.3 Register Map Utilities

In addition to system-level infrastructure, utilities were created for interfacing AHB with simpler register-based components. Two examples are worth noting:

- **ahb_rw_regs.sv:** a simple module which generates a register file with a parametrisable number of registers. The register values are also outputted via a static bus. This is useful for creating configuration registers, but unsuitable for any read-only status/data registers.
- **ahb_to_csr.sv:** a more flexible module which handles AHB decoding and pipelined access to provide a control-and-status register interface. Listing 5.5 shows the module signature with inline comments explaining the interface. This module is highly flexible and is used in most of the components to provide control and status registers, as it accommodates read/write side effects.

```

1 module ahb_to_csr #(
2     parameter NUM_REGS    = 4,
3     localparam DATA_WIDTH = 32, // localparam - not configurable
4     localparam RELEVANT_ADDR_WIDTH = 2+$clog2(NUM_REGS) // The number of
        bits of the address that are relevant for decoding which register
        is being accessed. The upper bits are ignored. Add two bits for
        word-alignment.
5 ) (

```

```

6      input                HCLK,
7      input                HRESETn,
8
9      // AHB connects using the standard 'ahb_intf_s' interface
10     ahb_intf_s.slave AHB_IF,
11
12     // CSR interface is simpler
13     output                access    [NUM_REGS-1:0],
14     // access[n] will pulse when register number n is read or written. May
15     // be used to trigger side-effects
16     output                write,
17     // (shared) 1 indicates that the accessed register is being written to
18     // . 0 means the register is being read
19     output [DATA_WIDTH-1:0] wdata,
20     // (shared) data being written. Invalid during read transfers
21     input  [DATA_WIDTH-1:0] rdata    [NUM_REGS-1:0],
22     // rdata[n] is the data being read from register n. May be ignored
23     // during write transfers, but it is simplest to just always drive the
24     // read data.
25     input                error,
26     // only valid when ready is 1. Can be tied to 0 if no errors are
27     // expected or detectable
28     input                ready
29     // indicates the transaction is done. Usually tied to 1 but can be
30     // used to insert wait states if necessary, though this is not
31     // recommended: if a slave requires wait states you should consider a
32     // more sophisticated AHB connection that takes advantage of the
33     // pipelining
34 );

```

Listing 5.5: Module signature of an `ahb_to_csr` module which translates the AHB protocol to a minimal interface for simple control/status registers.

5.4 Memory Map

All components on the SoC use a shared memory map, addressable via the AHB. The 32-bit address space provides more than sufficient address space.

Addresses were allocated such that the relevant SoC component could be identified (to be selected) using only the most significant bits. A high-level view of the address spaces allocated to each component is shown in figure 5.3.

Note that the address space is considered byte addressed. Most components on the SoC do not support non-word-aligned transactions, so the final two bits of the address must always be 00.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0					
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Word Address												0	0	ROM					
0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	Word Address												0	0	RAM				
0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Reg	Byte	GPIO					
0	0	0	0	0	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Reg	0	0	DMAC					
0	0	0	1	0	0	0	0	MVU #				0	0	Bank #				Address								W	0	0	MVU Data							
0	0	0	1	0	0	0	1	MVU #				0	0	Address (Line)								Address (Word)								0	0	MVU Weights				
0	0	0	1	0	0	1	0	MVU #				0	0	0	0	0	Address (Line)								Address (Word)								0	0	MVU Biases	
0	0	0	1	0	0	1	1	MVU #				0	0	0	0	0	0	Address (Line)								Address (Word)								0	0	MVU Scalers
0	0	0	1	1	0	0	0	MVU #				0	0	0	0	0	0	CSR Address												0	0	MVU CSR				

Figure 5.3: Overview of the allocated addresses for each component in the SoC. 0s and 1s indicate bits used to identify the component, while cells in white can be varied to access different addresses within one component. Addresses not mapped here will read invalid data and writes will be ignored.

A nybble is a group of four consecutive bits. Addresses with a ‘1’ in the first nybble are allocated to the MVU, while other components have a ‘0’ in the first nybble. The second nybble identifies the specific peripheral being accessed. A number of bits must always be zero because the address space is significantly larger than required. The remaining bits are used to address specific registers or memory lines within components.

To ensure scalable and easily maintainable address decoding logic, this memory map is only used in a single location in the design, at an `ahb_decoder` module in the SoC top. Similarly, firmware does not explicitly use addresses, but symbolic names which can be easily reassigned if the memory map changes in future development.

5.5 DMA Design

5.5.1 Design

The AHB provides a means for the core to read and write data from or to any peripheral. However, transferring large amounts of data will require the core to perform a large sequence of these operations, which is highly inefficient resource utilisation for a core capable of

more sophisticated processing. A direct-memory-access controller (DMAC) is the standard solution to this problem. It acts both as a *master* on the AHB — controlling transactions between other peripherals — and as a *slave* - receiving instructions from the core.

A simple DMAC was created for this project. It has the following control registers which may be written by the core to perform DMA operations:

- Source base address
- Destination base address
- Transfer size (in words)
- A “trigger” bit to begin the transfer
- Increment enables for both source and destination addresses (i.e. should the address be incremented after each word is transferred. For example, when reading from an I/O peripheral, it may be desirable to read continuously from the same address, which will provide new data on each access.)
- A status register (for reading status and clearing IRQs)

The DMAC simply alternates between reading data from source (storing the data in an internal buffer) and writing that buffer to the destination. A finite-state-machine orchestrates this process, with a counter that decrements on each completed transaction to track the remaining transfer size.

Due to the pipelined nature of the AHB, the DMAC is generally either:

- In the **address** phase for the source, while in the **data** phase for the destination, or
- Vice versa

The FSM reflects this – its state transition diagram is shown in Figure 5.4.

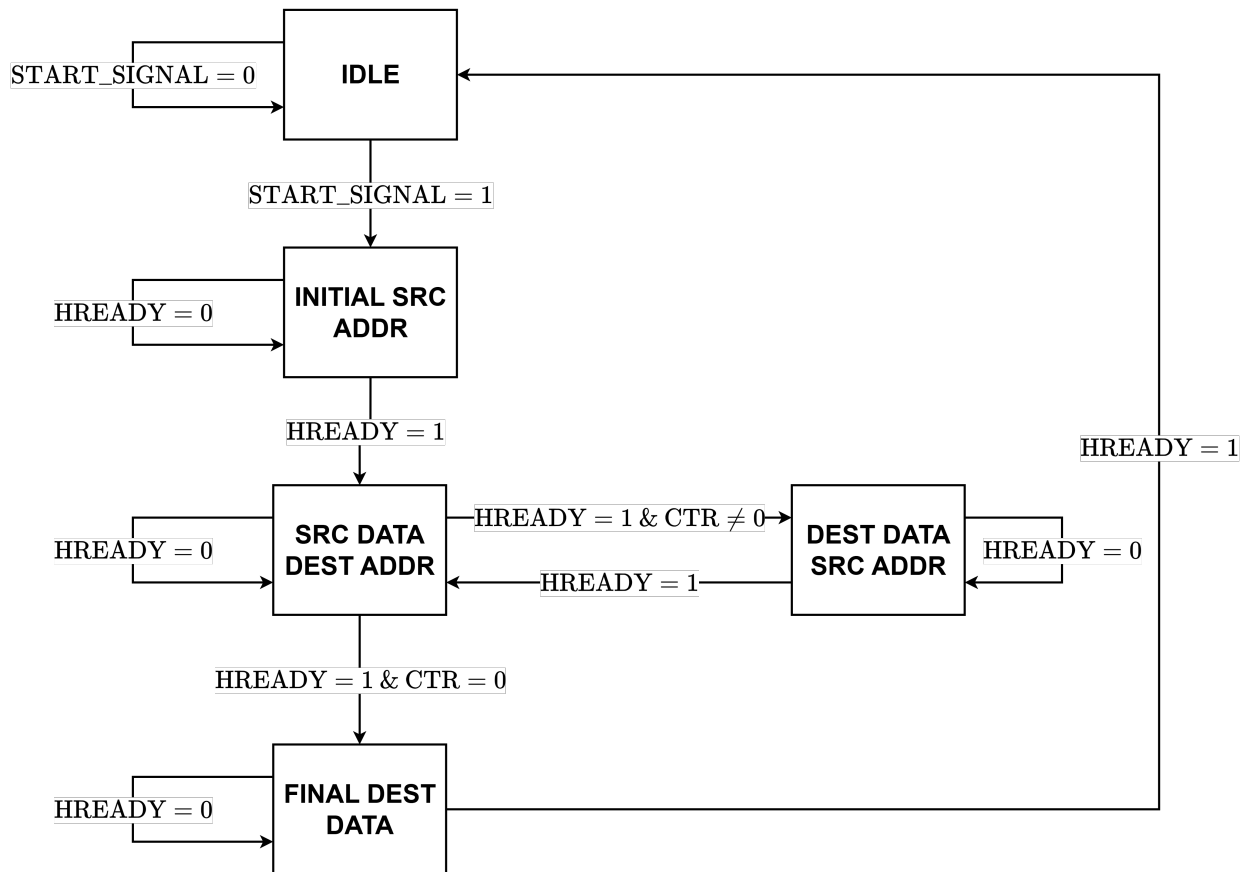


Figure 5.4: State transition diagram for the DMAC FSM.

The AHB signals are assigned using combinational logic based on the state of the FSM. When the state reaches “FINAL_DEST_DATA” and “HREADY”, an IRQ is raised to signal to the core that the transfer is complete.

There is a risk that an incorrectly configured DMAC transfer could cause the DMAC to attempt to overwrite its own configuration, resulting in unpredictable behaviour. Furthermore, an SoC architect may wish to restrict the DMAC’s access to certain region of memory for security purposes. For these reason, the DMAC includes a simple protection mechanism: two parameter are provided `BAD_ADDR_SPACE_VALUE` and `BAD_ADDR_SPACE_MASK`. If the DMAC is about to attempt a write to an address where `(address & BAD_ADDR_SPACE_MASK) == BAD_ADDR_SPACE_VALUE`, the DMAC will instead set an “error” bit in its status register, and return to the idle state (behaviour not shown on the FSM diagram).

5.5.2 Verification

The DMAC was verified with a simple testbench as follows:

1. The DMAC is instantiated and connected to two AHBs: `config_intf` and `dmac_bus_intf`.
2. `config_intf` will be driven by the testbench to control the DMAC.
3. `dmac_bus_intf` is connected to a simple AHB slave with read/write memory.
4. Test data is loaded into the slave memory by the testbench.
5. A series of AHB writes configurew the DMAC to transfer the test data to another address in memory.
6. The testbench awaits an interrupt from the DMAC.
7. The testbench verifies that the data at the destination address matches the test data.

The testbench is not comprehensive and does not verify the behaviour of non-trivial edge cases, but its typical use case has been verified which is sufficient for the purposes of this project.

5.6 AHB Arbitration

Including a DMA controller requires that the AHB can be controlled either by the Ibex core or the DMA controller. An AHB arbiter is included to ‘arbitrate’ control of the bus.

The AHB standard specifies that signals “HBUSREQ” and “HGRANT” be used by AHB masters to request control of the bus and have it granted by the arbiter. However, the AHB wrapper around the Ibex core implements AHB-Lite, which does not provide these signals. This made existing arbiter non-viable. Instead, a “transparent” AHB arbiter was design for this project, where the bus request is implied by the “HTRANS” signal, which indicates a master wishes to perform a transfer. The word “transparent” refers to the fact that the masters are not aware of the presence of the arbiter; if they request a transaction and the bus is in use, the HREADY signal to the master will be delayed. Masters are unable to differentiate between delays caused by the slave being busy or by the bus being busy, but this is not generally necessary in any case.

A conceptual diagram is shown in figure 5.5.

A flip-flop tracks the current “owner” of the control signals on the bus, and the data signals on the bus (on an AHB, the control signals and data are not sent simultaneously). Bus signals are multiplexed based on the state of these flip-flops. The HREADY signal that returns to each master is also gated depending on the state of the arbiter. Additionally, there are sets of flip-flops to store response data (HRESP and HRDATA), which is necessary in case a master is not granted its HREADY signal until some time after the slave believes the transaction is complete.

A static priority scheme gives the Ibex core higher priority than the DMAC, allowing the

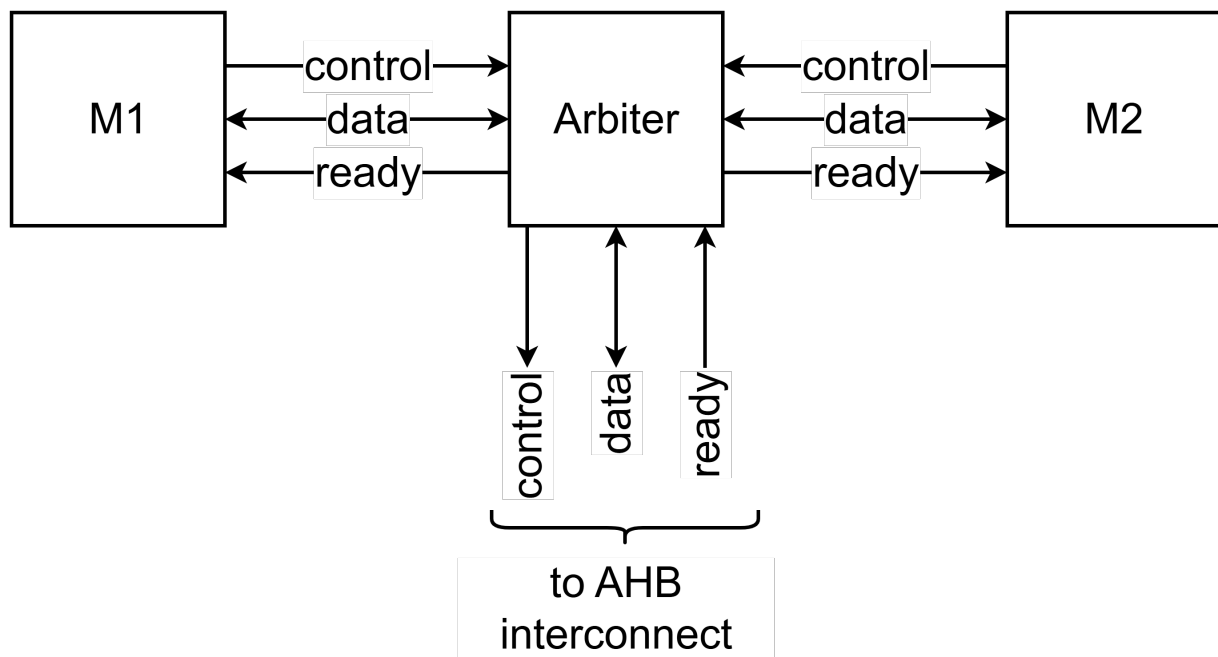


Figure 5.5: Conceptual diagram of the transparent AHB arbiter. Two AHB masters are connected to the arbiter, which drives the master signals for the interconnect. Control signals go from master → arbiter → interconnect → slave, while data flows in both directions. The HREADY signals allow downstream logic (slaves, or the arbiter) to delay the master if the bus is busy (‘stall’ a transaction).

core to continue to operate effectively while large data transfers complete in the background.

Listing 5.6 shows the SystemVerilog module signature for the AHB arbiter. Three AHB ‘master’ interfaces are provided (as described in section 5.3). Two are driven by masters believing they are the sole master. The arbiter drives a single ‘interconnect interface’ which can be connected to the rest of the system.

```
1 module ahb_transparent_arbiter (  
2     input HCLK,  
3     input HRESETn,  
4  
5     ahb_intf_m.interconn ahb_if_m1, // first master interface  
6     ahb_intf_m.interconn ahb_if_m2, // second master interface  
7     ahb_intf_m.master ahb_if_mi // interconnect master (driven by the  
8         arbiter)  
9 );
```

Listing 5.6: The SystemVerilog module signature for the transparent AHB arbiter. Two AHB ‘master’ interfaces are provided, each master believing they are the sole master. The arbiter drives a single ‘interconnect interface’ which can be connected to the rest of the system.

Figure 5.6 shows a typical example of the arbiter’s behaviour. Two masters (M1, M2) are contending for control of the bus. Each has various control signals (e.g. HADDR, HTRANS) and data signals (e.g. HRDATA and HWDATA). “MI” represents the ‘interconnect master’ which is mastered by the arbiter. M1 may represent a DMA with a series of continued back-to-back transactions. M2 is the higher-priority master (e.g. the CPU core) – at cycle 6, it requests control of the bus and is immediately granted by the arbiter. The master M1 is unaware that it has lost control of the bus, but its HREADY signal is delayed as if backpressure were being asserted by the slave. Due to the pipelined nature of the AHB protocol, control signals and data signals may be multiplexed from different masters on the same cycle (such as at cycles 6 and 7 in the figure).

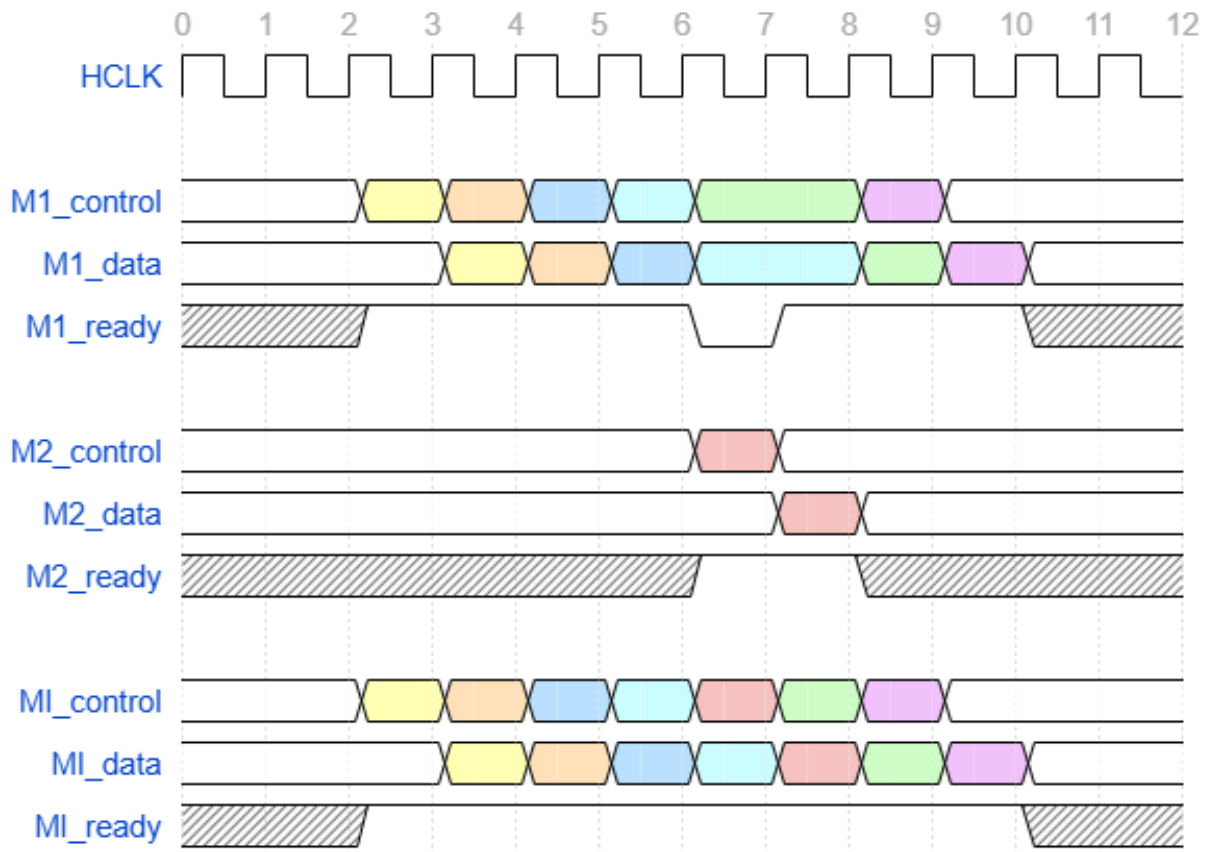


Figure 5.6: Example of a typical behaviour of the arbiter. White waveforms indicate idle states, while non-idle transactions are colour coded. Note that control signals and data signals are sent on different cycles under the AHB protocol. MI represents the arbitrated bus. Master 1 (M1) is performing a series of transactions and is interrupted by Master 2 (M2). M1 is unaware of the interruption, but is delayed using its 'ready' signal. After M2 completes its transaction, M1 can continue. The downstream AHB slaves may also delay the 'ready' signal, which the arbiter will pass on to both masters.

Chapter 6

System Review

6.1 Integration Testing

A number of end-to-end integrations tests were created to verify the SoC components were integrated correctly. These maximised the breadth of their coverage to catch any potential integration issues:

- Firmware was written in C to configure the hardware being tested
- The firmware was compiled using the flow described in section 5.2.1
- The SoC was simulated with the firmware loaded into instruction memory
- The firmware would write configuration to the hardware under test and read back statuses, then write special status code to GPIO outputs
- The testbench would monitor GPIO outputs to determine if the test passed or failed, and log the results

Tests were used during development to monitor for functional regressions introduced. There were four main regression tests:

- **DMAC Test:** Load random data into a location in memory. Configure the DMAC to copy this data to another location. Wait for the DMA IRQ. Read back the data and verify it is correct.
- **IRQ Test:** Configure IRQ handlers for each IRQ which write the IRQ number to GPIO output. The testbench triggers each IRQ in turn (using SystemVerilog `force` statements) and verifies the correct IRQ number is written to GPIO output.
- **MVU Test:** Load random data and weights into the MVU memory. Configure the MVU to perform a convolution. Wait for the MVU IRQ. Read back the output and verify it is correct.
- **AHB Infrastructure Test:** Read-only data is included in the program memory. The firmware reads this data and verifies it matches expected values. Then random locations in RAM are written and read to, including the first and last addresses.

These test cases cover the basic usage for every component of the SoC, and any integration issues would likely cause a test failure. However, edge cases are **not** covered in these regression tests, but rather by individual tests run on each component.

6.2 FPGA Implementation

The SoC design was synthesised for FPGA implementation. Vivado was used as the synthesis/implementation tool, accessed via FuseSoC which provided a wrapper to set up and configure the project, managing source files and dependencies.

An AMD Zynq Ultrascale+ MPSoC was targetted.

Care was taken to ensure that block RAMs were used to implement internal memory banks in the MVU, RAM, and instruction memory, as block RAMs provide significantly higher density and lower power consumption than relying on flip-flops in the FPGA fabric. To accomplish this, the Vivado Block Memory Generator tool was used (from the IP catalog)

which allows block RAM to be configured with the desired width, depth, and interface. The generated memory modules were instantiated directly, giving the synthesis tool a clear indication that they should be implemented using block RAMs.

The FPGA utilisation of the design is discussed in chapter 7.

6.3 Power Estimation

Power consumption is a critical target for optimisation in edge computing and embedded devices. In this section, the techniques for estimating the power consumption of the SoC are described.

Vivado provides power estimates for post-implementation designs. However, under default settings, Vivado’s power estimates are based on static analysis and generic assumptions about signal activity. These estimates do not account for the actual usage patterns or switching activity of the design in a real application, which gives very little confidence in the accuracy of the power predictions.

To improve accuracy, Vivado supports the use of Switching Activity Interchange Format (SAIF) files. A SAIF file captures detailed information about the real switching activity of signals during a simulation. By running a simulation that exercises the design—using representative input data and workloads—one can generate a SAIF file that reflects realistic usage.

For this project, various workloads were simulated with differing firmware. For each one, the switching activity at the top-level of the CPU and the MVU was captured in a safe file. The power analysis procedure is divided into the following phases:

1. **Setup:** The firmware is compiled.
2. **Pre-firmware simulation:**
 - The design is elaborated by Vivado.

- The firmware is loaded into the instruction memory.
 - Random data is written into the data and weight memory of the MVU.
 - The ROM is ‘peeked’ and logged for verifying it has loaded correctly.
 - The reset is released, handing control to the firmware.
3. **Firmware initialisation:** The firmware enables the appropriate interrupts and writes random data to RAM for the DMAC to transfer.
 4. **Firmware configuration:** The firmware configures the MVU, DMAC, and CPU to generate the desired workload. It then sends a signal to the testbench that the workload is about to begin.
 5. **Workload:**
 - The TCL script opens a SAIF file and instructs Vivado to begin logging switching activity.
 - The simulation resumes and the firmware performs the workload.
 - During this time, the testbench will monitor the GPIO output for signals indicating the test is complete.
 6. **Termination:** When the firmware writes 0xBEEF to GPIO, the testbench halts and the TCL script will close the SAIF file.
 7. **Reporting:** The TCL script will then open the implemented design and use the `read_saif` and `report_power` commands to generate a power report based on the captured switching activity.

To give accurate power consumption figures, switching activity was recorded for top-level signals of the CPU, MVU, and overall DUT, as well as the on the SoC AHB to provide information about activity in other components. Additionally, the data and weight signals inside the MVU were logged to guarantee the power estimation tool had accurate information

about the switching activity of these signals, which are expected to be the most active and power-hungry parts of the design.

A number of different workloads were simulated including different combinations of:

- Number of MVU cores active (0, 1, or all 4)
- CPU status (asleep or performing arithmetic calculations)
- DMAC activity (idle or performing transfers)

CPU activity was simulated by running a loop that would generate a random value using a software-implemented PRNG, multiply it by itself, and store the result in memory. This was intended to simulate a realistic workload with a combination of memory accesses and arithmetic operations. The PRNG ensures switching activity is sufficiently random.

DMAC activity was simulated by performing a large transfer of 1,024 words from one location in RAM to another. Each time the transfer complete, an interrupt was generated, and the ISR would immediately trigger another transfer, creating a continuous stream of DMAC activity.

Power consumption results are presented in chapter 7.

6.4 Neural Network Firmware

6.4.1 C API

To accommodate the use of the SoC as a research platform for software development, a C API was developed to provide a convenient interface for firmware developers to interact with the SoC's hardware features. This API abstracts away the details of memory-mapped register access and provides higher-level functions for common operations such as configuring the MVU for convolutions and adjusting its precision. Signatures of the C functions provided are shown in listing 6.1.

```

1 void mvu_configure_conv_2d(
2     uint8_t mvu_id,
3
4     // Assume square kernel and image
5     uint8_t kernel_side_length,
6     uint8_t input_data_side_length,
7
8     // Set base addresses
9     uint32_t input_data_addr,
10    uint32_t weight_addr,
11    uint32_t output_data_addr,
12    uint8_t output_mvu_id,
13
14    // Stored precision is needed to determine the jump schedule
15    // even if we are only using part of the precision
16    uint8_t weight_max_precision,
17    uint8_t input_data_max_precision
18 );
19
20 void mvu_configure_precision(
21     uint8_t mvu_id,
22     uint8_t weight_precision,
23     uint8_t input_precision,
24     uint8_t output_precision,
25     uint8_t output_data_msbs_to_cut
26 );
27
28 void mvu_trigger();

```

Listing 6.1: Signature of the C methods provided for interacting with the MVU

The former two functions write to the appropriate configuration registers in the MVU register map to set up a convolution operation. The precision of the operation can be controlled independently and modified immediately before triggering the operation, allowing for flexible, dynamic-precision inference. The latter function writes to the control register to trigger the MVU to begin its operation.

6.4.2 Proof of Concept

The ultimate objective of this project was to prove that dynamic-precision neural network inference could be efficiently implemented on an SoC using bit-serial arithmetic. To this end, a proof-of-concept firmware program was developed to perform a series of convolutions with a ReLU activation function inbetween.

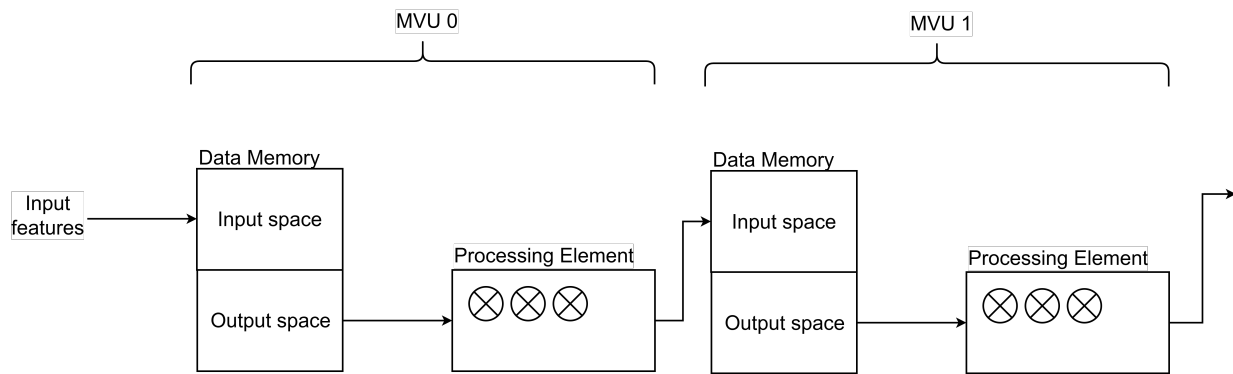


Figure 6.1: The MVUs can write into each other’s memory, allowing them to be daisy-chained together to perform a series of calculations.

The program assigns one layer of the convolution to each MVU. Each performs a convolution with a 3×3 kernel, beginning with 20×20 input data that is gradually reduced to 18×18 , 16×16 , 14×14 , then 12×12 .

The output from one calculation is written to the data memory of the next. For this reason, each MVU then has its data memory divided into two sections: one for the input data to its current calculation, and one for the previous MVU to output data to, as shown in figure 6.1. Once all four MVUs have completed a convolution, the input pointer and output pointers are swapped and the next calculation can begin. Effectively there is a ‘pipeline’ of MVUs.

The first MVU in the chain is fed with input data - for the purposes of this project, random data is generated by the firmware to be fed into the first MVU.

All four MVUs were configured in this way and the chain of convolutions verified against a Python implementation of the same network.

In a practical neural network, assigning one neural network layer to each MVU is not practical for large networks, so more sophisticated firmware would be necessary to schedule the MVUs to perform different layers. However, this proof-of-concept demonstrates the feasibility of the approach, and the software suite and firmware API provided enables rapid development of more sophisticated concepts.

Part III

Conclusion

Chapter 7

Results and Research Contribution

7.1 FPGA Resource Utilisation

Table 7.1 shows the utilisation of FPGA resources for the SoC design when targetted to a Xilinx Zynq Ultrascale+ MPSoC. The design was synthesised with a clock frequency of 150 MHz.

Resource	DyNNSoC Utilisation		Utilisation for one MVU	
	#	%	#	%
BRAM (36 kb)	715	78.45%	176	19.30%
BRAM (18 kb)	1	0.05%	0	0%
DSPs	257	10.20%	64	2.54%
LUTs	99,617	36.35%	23,456	8.56%
FFs	77,723	14.18%	17,152	3.13%

Table 7.1: FPGA resource utilisation of the whole DyNNSoC, or for one MVU core.

7.2 Power Consumption

Table 7.2 shows the power consumption of the SoC under various loads. Measurement methodology was discussed in section 6.3

Table 7.3 shows the power consumption of a single MVU when active vs. inactive. While there is a significant difference between the two scenarios, the inactive power consumption

DMAC Status	CPU Status	MVUs Active	Total Power
Active	Asleep	0	1,311 mW
Active	Asleep	1	1,372 mW
Active	Asleep	4	1,543 mW
Active	Active	4	1,546 mW
Idle	Asleep	4	1,515 mW

Table 7.2: The power consumption of the SoC under various loads.

Status	Total Power
Active	329 mW
Inactive	264 mW

Table 7.3: The power consumption of a single MVU when active vs. inactive

remains very high. Despite the lack of switching activity, power is still consumed due to clock signals toggling at flip-flop inputs, and due to leakage in transistors. The MVU is a large, power-hungry design which may often be idle in some applications, making it a good candidate for further power optimisation by introducing clock gating and power gating – i.e. the power supply and clock signal should be disabled when the MVU is inactive. This is a potential avenue for future work.

Status	Total Power
Active	87 mW
Inactive	84 mW

Table 7.4: The power consumption of the CPU core when active vs. asleep

Similarly, table 7.4 shows the power consumption of the CPU core when active vs. asleep. There is extremely little difference. However, the Ibex core already has an option to enter low-power mode with clock gating, which was not enabled during this project. Enabling this feature would likely yield a much more significant reduction in power consumption when the CPU is asleep, and is another potential avenue for future work.

7.3 Discussion

The power consumption results show a drastic reduction from the original design in [4], which consumed 2.6 W per MVU when active. However, this should **not** be construed as a major performance improvement. In fact, this is a consequence of two changes: the clock speed was reduced from 250 MHz to 150 MHz to accommodate the needs of the Ibex core, and the design was synthesised for an FPGA with 16 nm process technology, which is more power efficient than the 28 nm process used in [4]. Neither design was created with power optimisation in mind, and there remains considerable scope for future work in this area. Rather, this project focused on addressing functional issues with the design, creating supporting software and firmware, and integrating with a broader system of hardware.

The project has demonstrated the conceptual viability of using bit-serial arithmetic to perform dynamic-precision neural network inference on an SoC. Further, it provides an open-source code platform for future research in this area to build on. At this time, this is the only platform known to employ bit-serial arithmetic to accommodate dynamic quantisation in NN inference.

The computation time required for a convolution operation is proportional to the precision used for the parameters and the input data. Accordingly, a reduction from 16-bit precision to 8-bit precision yields a $\times$ reduction in 4 compute time, and in energy consumed

per inference.

Chapter 8

Suggestions for Future Work

There are a number of avenues for further exploration in this project, both in terms of technical development improving on the existing design, and research questions invited by the work done in this project.

8.1 Scope for Technical Improvements

The most obvious technical improvement to the design is to implement clock gating and power gating for the MVU cores. As shown in Table 7.3, the inactive power consumption of the MVU is very high, and this would be a significant improvement to the design.

There is also scope for power optimisation inside the MVU design – for instance, the DSPs used for multiplication are only necessary when the accumulation stage of the bit-serial operation is completed, but they are active at all times, despite being complex clouds of combinational logic that consume a lot of power. A flip-flop with an enable signal could be placed on the input of the DSPs to prevent unnecessary switching.

There is also scope for improvement on the C API which currently only provides minimal functions for convolution configuration, precision configuration, and triggering the MVU. In writing test firmware, it became apparent that reading and writing data in the transposed format required by the MVU would be a common and very cumbersome operation – this

could be implemented in the API, or with a dedicated hardware block attached to the MVU or DMAC. Finally, API methods should be implemented for matrix multiplication in addition to convolution, which is only a matter of modifying the values calculated for the AGU jump schedules.

8.2 Scope for Continued Research

Previous research [15] [11] has demonstrated that dynamic-precision is a promising approach to increasing the efficiency of neural network inference. This project provides a path for implementing this in practice, but no firm analysis has been done relating the power savings directly to inference accuracy loss. This would require a complex firmware stack and precise power analysis.

Additionally, while the computational savings offered by dynamic quantisation are reasonably clear, the algorithms for determining the optimal precision for each operation are not well-researched. A number of approaches are reviewed in section 2.2, but further research is necessary in this space. This may reveal novel approaches that would benefit from dedicated hardware support, which would be in the scope of this project.

Chapter 9

Ethics and Sustainability

The modern revolution in Artificial Intelligence, spurred by Neural Networks, has exacerbated concerns about the sustainable implications of the modern computing paradigm. In

this section, both positive impact of this project and potential ethical challenges are discussed and related to the UN Sustainable Development Goals (SDGs).

Water and energy consumption

Training and use of artificial intelligence models (built largely on neural networks) has come to be recognised in recent years as an environmental concern due to the enormous electricity consumption, and associated consumption of clean water for cooling. A 2025 study [14] found that AI demand may be responsible for 4.2-6.6 billion cubic meters of water consumption in 2027, which is half the consumption of the entire United Kingdom. Regarding energy, there is limited specific calculations on the electricity demand of AI, as it may be difficult to separate from other data centre demands, but the scale of the energy issue is certainly similar [6].

These challenges motivate the continued research into novel approaches to reducing the energy footprint of artificial intelligence. Dedicated hardware accelerators have already demonstrated an enormous savings in energy, but the widespread deployment of dynamic neural networks may offer further substantial economies. Details of the energy savings offered by dynamic networks were explored in the literature review.

UN SDG 7 (Affordable and Clean Energy) has indicator 7.3.1 which targets a doubling of energy efficiency (energy consumption per unit GDP). It is not yet clear to anyone whether the increased economic productivity AI offers will justify the energy consumption by this metric, but improved efficiency of AI accelerators can only improve this.

Similarly UN SDG 6 (Clean Water and Sanitation) is impacted in a broader sense by the drastic increase in water consumption demanded by AI tools, and improved efficiency of AI accelerators may alleviate these stresses.

The Jevons Paradox

Unfortunately, the Jevons paradox is often said to apply to the argument above, which notes that the reducing the resource demands of AI will likely see a matching increase in overall demand for AI, ultimately placing more strain on the environment. These challenges

are beyond the scope of electronic engineering.

Ethical Considerations

There have been ethical concerns that the AI revolution will be focused in developed countries and exacerbate global inequalities in technology access. Because of this, this project is also motivated by a democratisation and academic focus on AI research. The code base itself is open-source and relies where possible on open source tools (FuseSoC, Onnx, Python).

In a broader sense this project contributes to the AI revolution ongoing which may have more complex ethical consequences spanning its potential for misuse, spreading misinformation or introducing biases, or a more concerns regarding human art and our way of life. Any judgement whether this entire research topic is ethically justified is far beyond the scope of this report.

Waveform diagrams were created using <http://wavedrom.com/>

Diagrams were created using <https://app.diagrams.net/>

References

- [1] Arm Ltd. *AMBA AHB Protocol Specification*. Issue C. Arm Ltd. 2021. URL: <https://developer.arm.com/documentation/ih0033/c>.
- [2] MohammadHossein AskariHemmat et al. “RISC-V barrel processor for deep neural network acceleration”. In: *2021 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE. 2021, pp. 1–5.
- [3] Mohammadhossein Askarihemmat et al. “BARVINN: Arbitrary precision DNN accelerator controlled by a RISC-V CPU”. In: *Proceedings of the 28th Asia and South Pacific Design Automation Conference*. 2023, pp. 483–489.
- [4] Oleksa Bilaniuk et al. “Bit-slicing FPGA accelerator for quantized neural networks”. In: *2019 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE. 2019, pp. 1–5.
- [5] *Chameleon SoC*. URL: https://github.com/shalan/Chameleon_SoC/tree/main.
- [6] Alex De Vries. “The growing energy footprint of artificial intelligence”. In: *Joule* 7.10 (2023), pp. 2191–2194.
- [7] Biyi Fang, Xiao Zeng, and Mi Zhang. “Nestdnn: Resource-aware multi-tenant on-device deep learning for continuous mobile vision”. In: *Proceedings of the 24th Annual International Conference on Mobile Computing and Networking*. 2018, pp. 115–127.

- [8] Yizeng Han et al. *Dynamic Neural Networks: A Survey*. 2021. arXiv: 2102.04906 [cs.CV]. URL: <https://arxiv.org/abs/2102.04906>.
- [9] Kaiming He et al. “Deep residual learning for image recognition”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2016, pp. 770–778.
- [10] Benoit Jacob et al. “Quantization and training of neural networks for efficient integer-arithmetic-only inference”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2018, pp. 2704–2713.
- [11] Qing Jin, Linjie Yang, and Zhenyu Liao. “Adabits: Neural network quantization with adaptive bit-widths”. In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2020, pp. 2146–2156.
- [12] Olof Kindgren. “A scalable approach to IP management with FuseSoC”. In: *1st Workshop on Open-Source Design Automation (OSDA)*. Vol. 5. 2019, p. 30.
- [13] Raghuraman Krishnamoorthi. “Quantizing deep convolutional networks for efficient inference: A whitepaper”. In: *arXiv preprint arXiv:1806.08342* (2018).
- [14] Pengfei Li et al. “Making AI Less ’Thirsty’”. In: *Commun. ACM* 68.7 (June 2025), pp. 54–61. ISSN: 0001-0782. DOI: 10.1145/3724499. URL: <https://doi.org/10.1145/3724499>.
- [15] Zhenhua Liu et al. “Instance-aware dynamic neural network quantization”. In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2022, pp. 12434–12443.
- [16] lowRISC. *Ibex RISC-V Core*. URL: <https://github.com/lowRISC/ibex>.
- [17] Tamador Mohaidat and Kasem Khalil. “A survey on neural network hardware accelerators”. In: *IEEE Transactions on Artificial Intelligence* 5.8 (2024), pp. 3801–3822.
- [18] Thomas O’Dea. *DyNNSoC*. URL: <https://github.com/thomas6785/DyNNSoC>.

- [19] Augustus Odena, Dieterich Lawson, and Christopher Olah. “Changing model behavior at test-time using reinforcement learning”. In: *arXiv preprint arXiv:1702.07780* (2017).
- [20] Tasnim Shahriar. *Comparative analysis of lightweight deep learning models for memory-constrained devices*. 2025.
- [21] Open Source. *RISC-V GNU Compiler Toolchain*. URL: <https://github.com/riscv-collab/riscv-gnu-toolchain>.
- [22] Surat Teerapittayanon, Bradley McDanel, and Hsiang-Tsung Kung. “Branchynet: Fast inference via early exiting from deep neural networks”. In: *2016 23rd international conference on pattern recognition (ICPR)*. IEEE. 2016, pp. 2464–2469.
- [23] Kuan Wang et al. “Haq: Hardware-aware automated quantization with mixed precision”. In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2019, pp. 8612–8620.
- [24] Qingyuan Wang et al. “DyCE: Dynamically Configurable Exiting for deep learning compression and real-time scaling”. In: *Future Generation Computer Systems* 171 (2025), p. 107837.
- [25] Zuxuan Wu et al. “Blockdrop: Dynamic inference paths in residual networks”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2018, pp. 8817–8826.
- [26] Jiahui Yu et al. “Slimmable neural networks”. In: *arXiv preprint arXiv:1812.08928* (2018).
- [27] Shuchang Zhou et al. “Dorefa-net: Training low bitwidth convolutional neural networks with low bitwidth gradients”. In: *arXiv preprint arXiv:1606.06160* (2016).

Bibliography

- [28] Krste Asanović and David A Patterson. “Instruction sets should be free: The case for risc-v. EECS Department”. In: *University of California, Berkeley, Tech. Rep. UCB/EECS-2014-146* (2014).
- [29] The Linux Foundation. *Open Neural Network Exchange*. 2021. URL: <https://onnx.ai/> (visited on 01/18/2026).
- [30] Graham Neubig et al. “DyNet: The Dynamic Neural Network Toolkit”. In: *arXiv preprint arXiv:1701.03980* (2017).
- [31] Ximeng Sun et al. “Dynamic network quantization for efficient video inference”. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 2021, pp. 7375–7385.
- [32] Andrew Waterman et al. “The RISC-V instruction set manual”. In: *Volume I: User-Level ISA, version 2* (2014), pp. 1–79.
- [33] Haichao Yu et al. “Any-precision deep neural networks”. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 35. 12. 2021, pp. 10763–10771.

AI Use Disclosure

This report was produced with the assistance of large language model (LLM) tools. These tools were used primarily for grammar and spelling checks (e.g. “read this section and identify spelling or grammatical errors”), as well as for its expertise with LaTeX syntax for formatting the report (e.g. “create a figure with two subfigures bee1.png and bee2.png”).

AI tools were used for technical work in only two places.

GitHub Copilot was used experimentally for the development of AHB infrastructure - this is outlined in the CREDITS.md file in the code repository. AI-generated RTL code was found to be problematic and was largely re-written manually. AI-generated testbenches were found to be more useful. To mitigate verification bias, testbenches and RTL were generated by different AI models in different context windows, without access to each others’ files. All high-level integration testing was done by a human to ensure specifications were interpreted correctly.

GitHub Copilot was also used to assist in creating firmware infrastructure. The firmware required a linker script and Makefile which are relatively standard and formulaic, only varying in the specific memory map and file names. These files use a cumbersome syntax that is unfamiliar to the author, and AI tools are well-known to perform very well at creating Makefiles. The firmware itself was written by hand, only the Makefile and linker script were generated with AI assistance.

All other work is the original work of this author except where the work of others is referenced and cited.